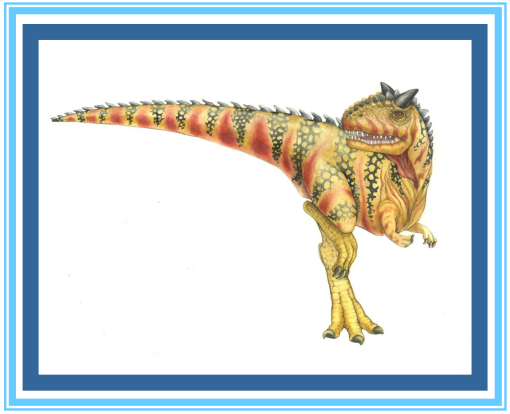
Chapter 5: Process Scheduling



Silberschatz, Galvin and Gagne ©2013



Chapter 5: CPU Scheduling

- 5.1 Basic Concepts
- 5.2 Scheduling Criteria
- 5.3 Scheduling Algorithms

OBJECTIVES

- To introduce CPU scheduling, which is the basis for multiprogrammed operating systems
- To describe various CPU-scheduling algorithms





5.1 Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- Most processes exhibit the following behavior:
- **CPU burst** followed by **I/O burst**
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- CPU burst distribution is of main concern

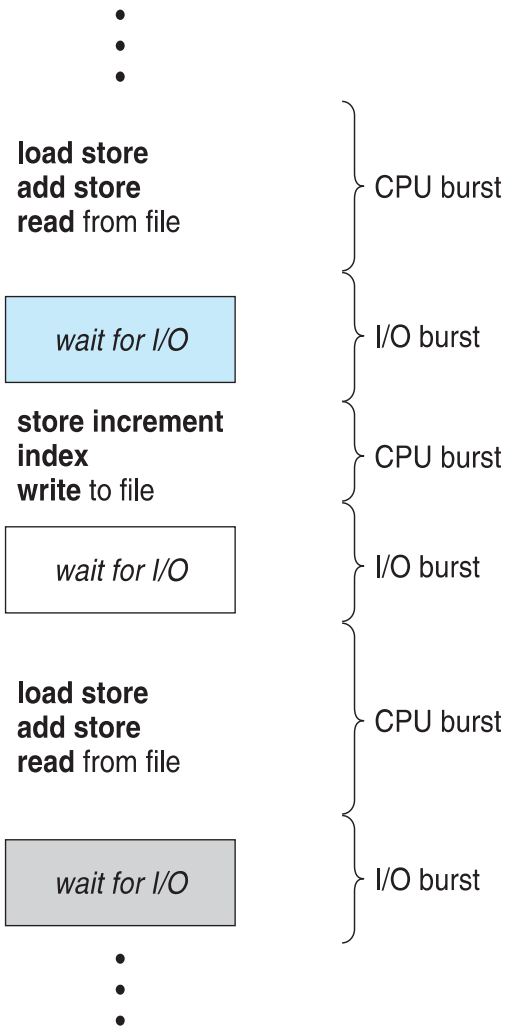


Figure 5.1 Alternating sequence of CPU and I/O bursts



CPU-I/O Burst Cycle

```
{
   printf("Enter first integer: ");
   scanf("%d", &b);

   y = b+(b*b)+1;

   printf("\n b+(b*b)+1 = %d", y);
}
```

(i) _____

(ii) _____

(iii) _____

Figure: Example part of C language programming



Histogram of CPU-burst Times

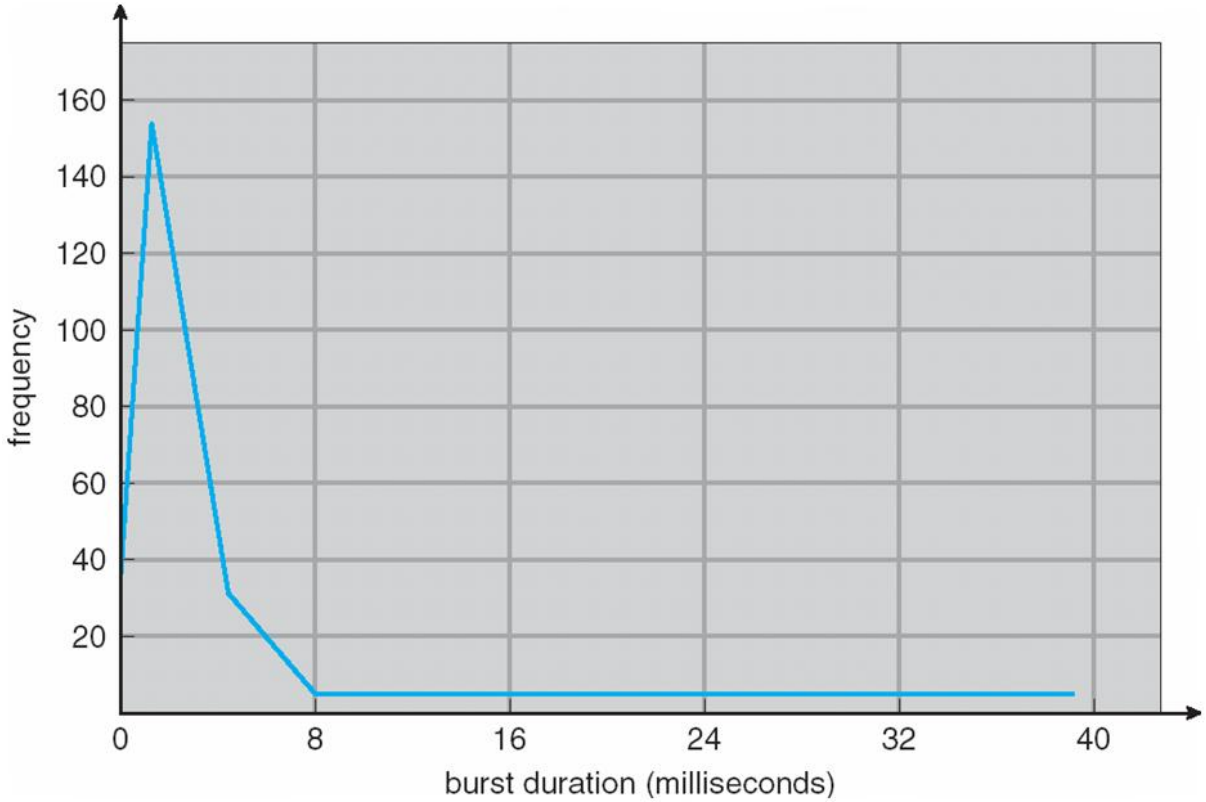
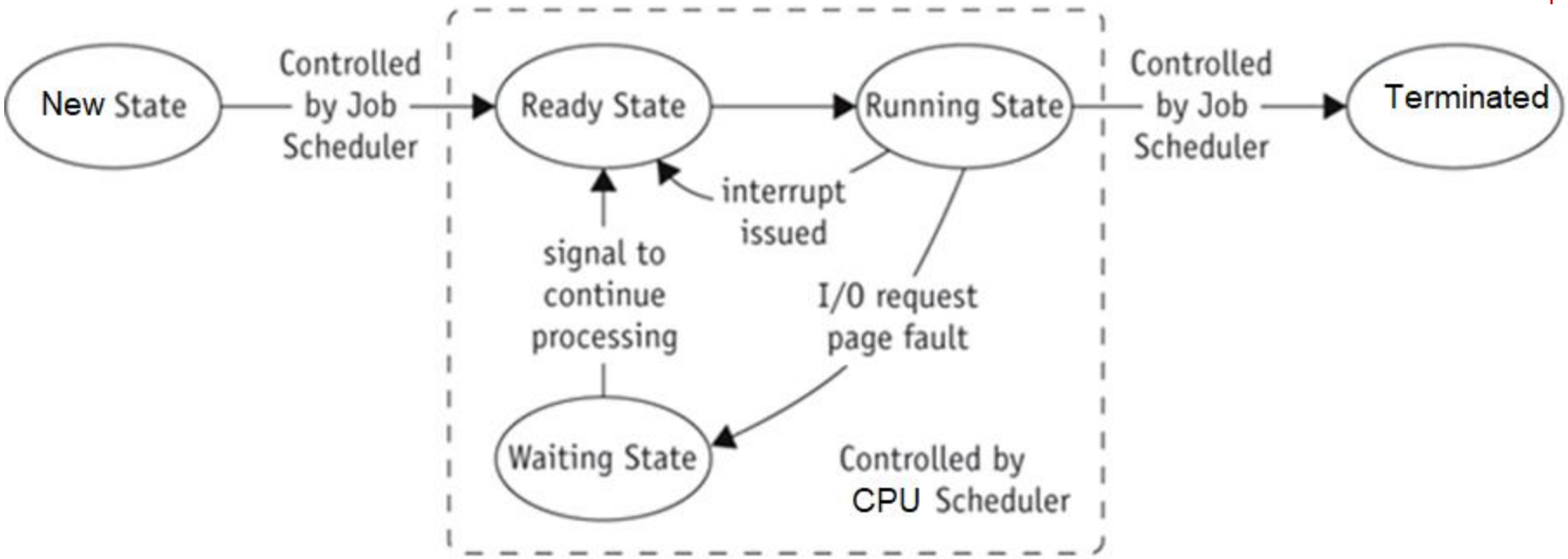


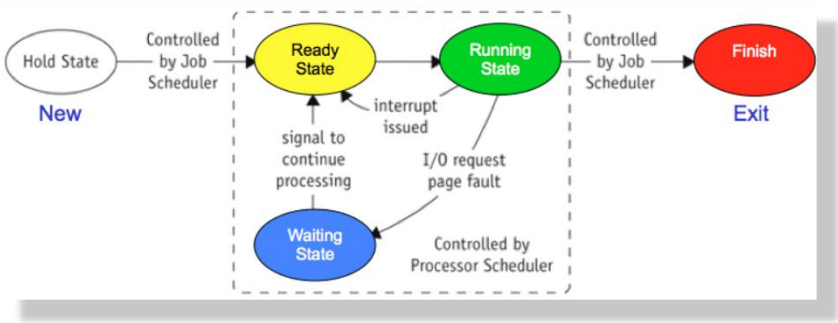
Figure 5.2



Job and Process Status

- Jobs move through the system.
- Job scheduler (long-term) vs CPU scheduler (short-term)





- User submits job (batch/interactive)
  - Job accepted
    - Put on HOLD and placed in queue
  - Job state changes from NEW to READY
    - Indicates job waiting for CPU
  - Job state changes from READY to RUNNING
    - When selected for CPU and processing
  - Job state changes from RUNNING to WAITING
    - Requires unavailable resources
  - Job state changes to TERMINATED
    - Job completed (successfully or unsuccessfully)

PCBs and Queuing

- Job PCB
  - Created when Job Scheduler (JS) accepts job
  - Updated as job executes
  - Queues use PCBs to track jobs
  - Contains all necessary job management processing data
  - PCBs linked to form queues (jobs not linked)
  - Manage queues using:
    - Process scheduling policies and
    - Process algorithms

(Continued Next)



CPU Scheduler

- Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed.
- The selection process is carried out by the **CPU scheduler**.
- The ready queue may be ordered in various ways.
- CPU scheduling decisions may take place when a process:
  1. Switches from running state to waiting state
  2. Switches from running state to ready state
  3. Switches from waiting state to ready state
  4. When a process terminates





Nonpreemptive Scheduling

- For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution.

- Under nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.





Preemptive scheduling

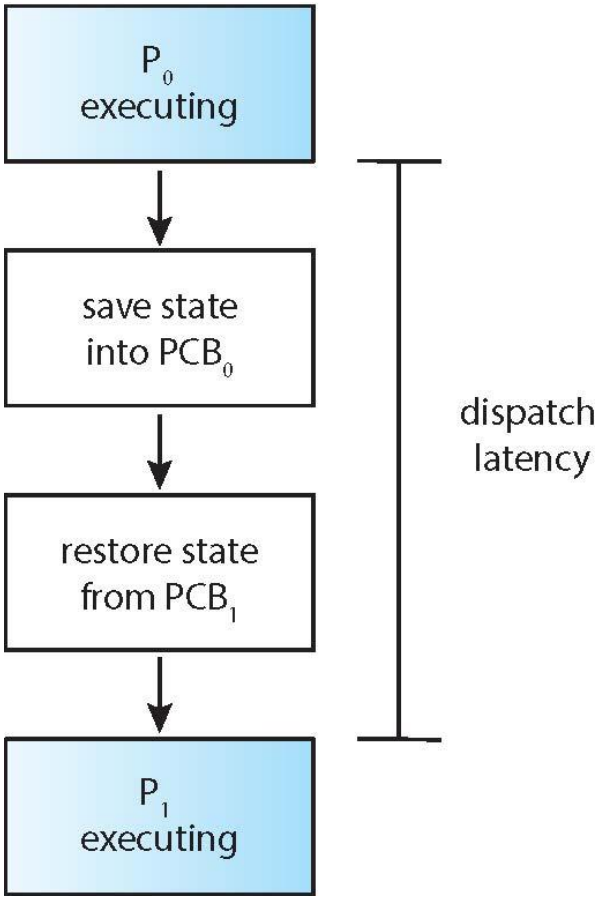
- There is a choice, however, for situations 2 and 3.
- Preemptive scheduling can result in race conditions when data are shared among several processes.
  - Consider the case of two processes that share data. While one process is updating the data, it is preempted so that the second process can run. The second process then tries to read the data, which are in an inconsistent state.
  - Consider preemption while in kernel mode
  - Consider interrupts occurring during crucial OS activities
- Virtually all modern operating systems including Windows, Mac OS X, Linux, and UNIX use preemptive scheduling algorithms.





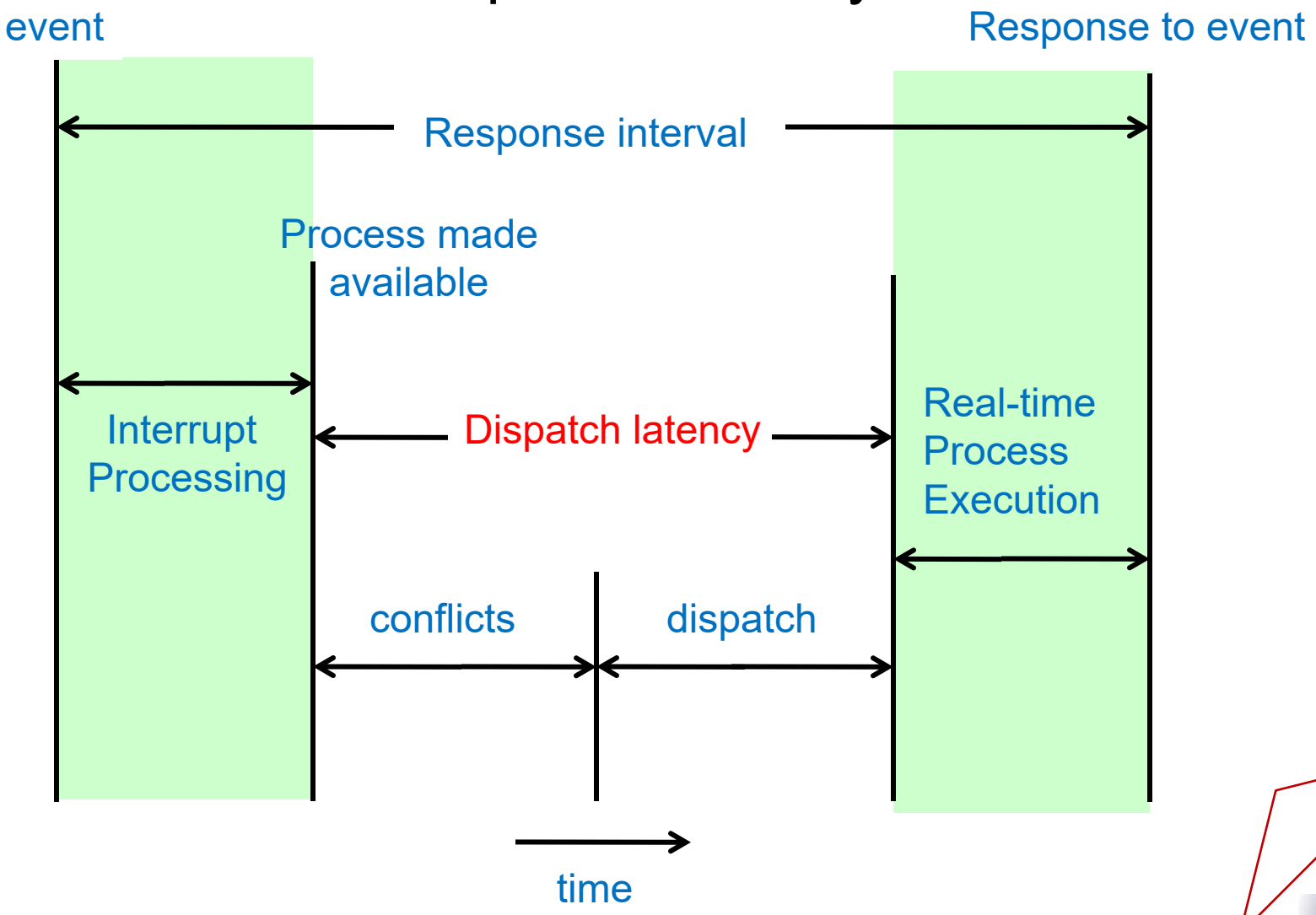
Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the CPU scheduler; this involves:
  - switching context
  - switching to user mode
  - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running





Dispatch Latency



Example:

Concurrent Execution of Processes (1/3)

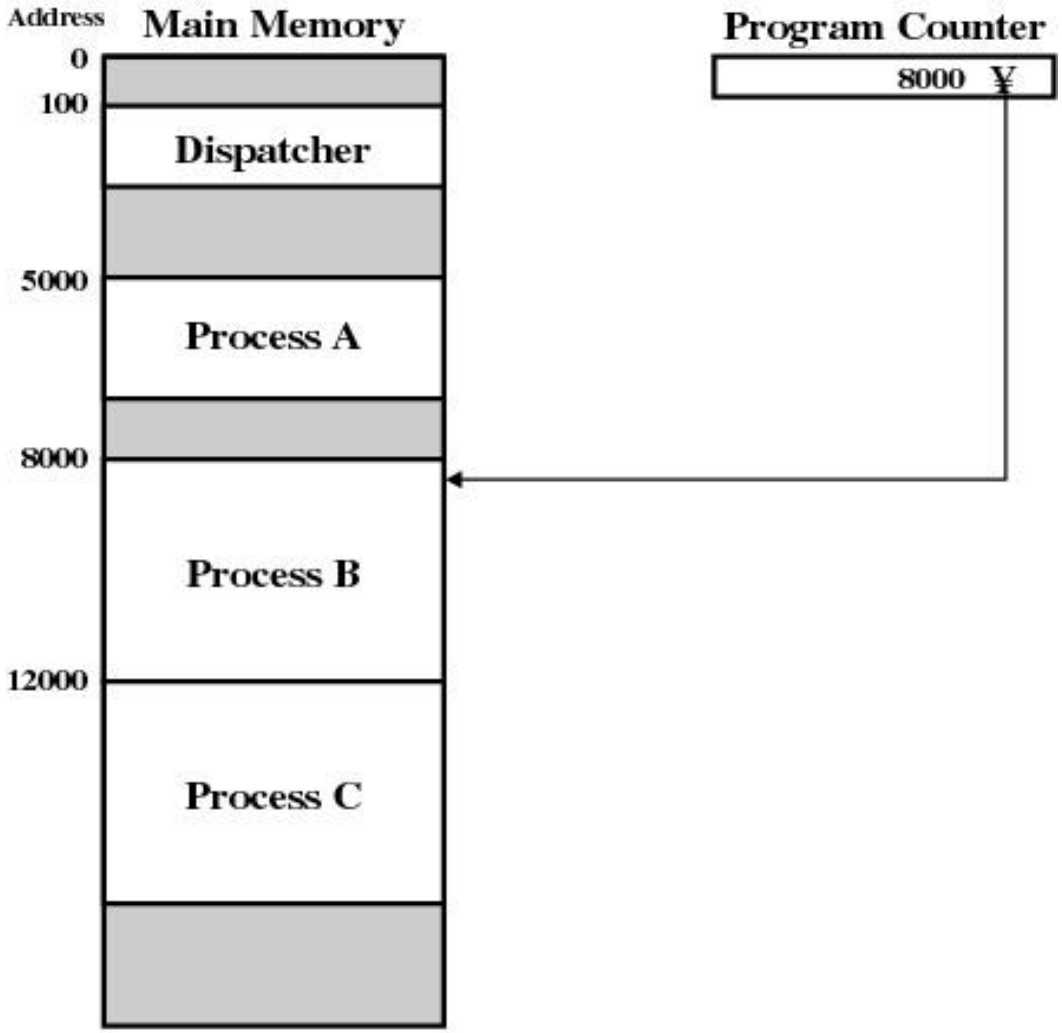
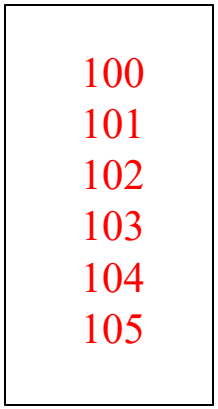
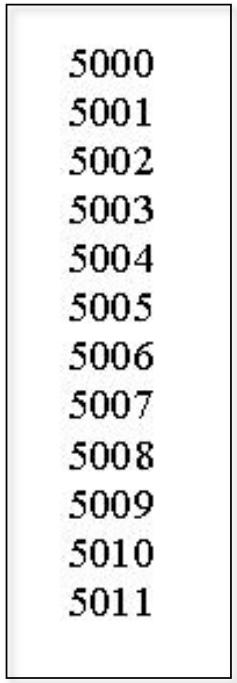


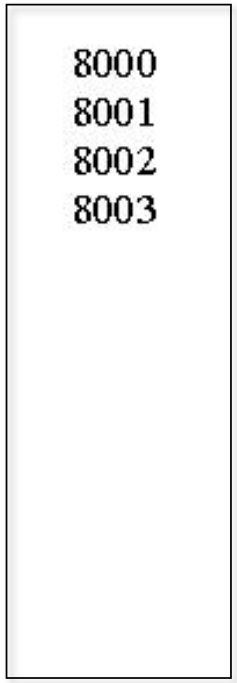
Figure 3.1 Snapshot of Example Execution (Figure 3.3) at Instruction Cycle 13



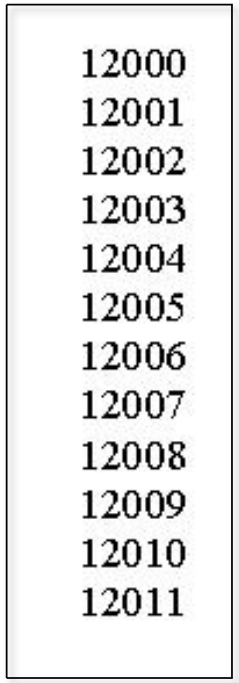
Dispatcher



(a) Trace of Process A



(b) Trace of Process B



(c) Trace of Process C

100 = Starting address of dispatch program.
5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C

Figure 3.2 Traces of Processes of Figure 3.1

Notes:

- **CPU time** or **instruction cycle** = 6 (each process)
- Assume all processes execute in sequence with no priority
- Shaded areas indicate execution of dispatcher process
- 1st & 3rd columns show number of instruction cycles
- 2nd & 4th columns shows address of instruction being executed

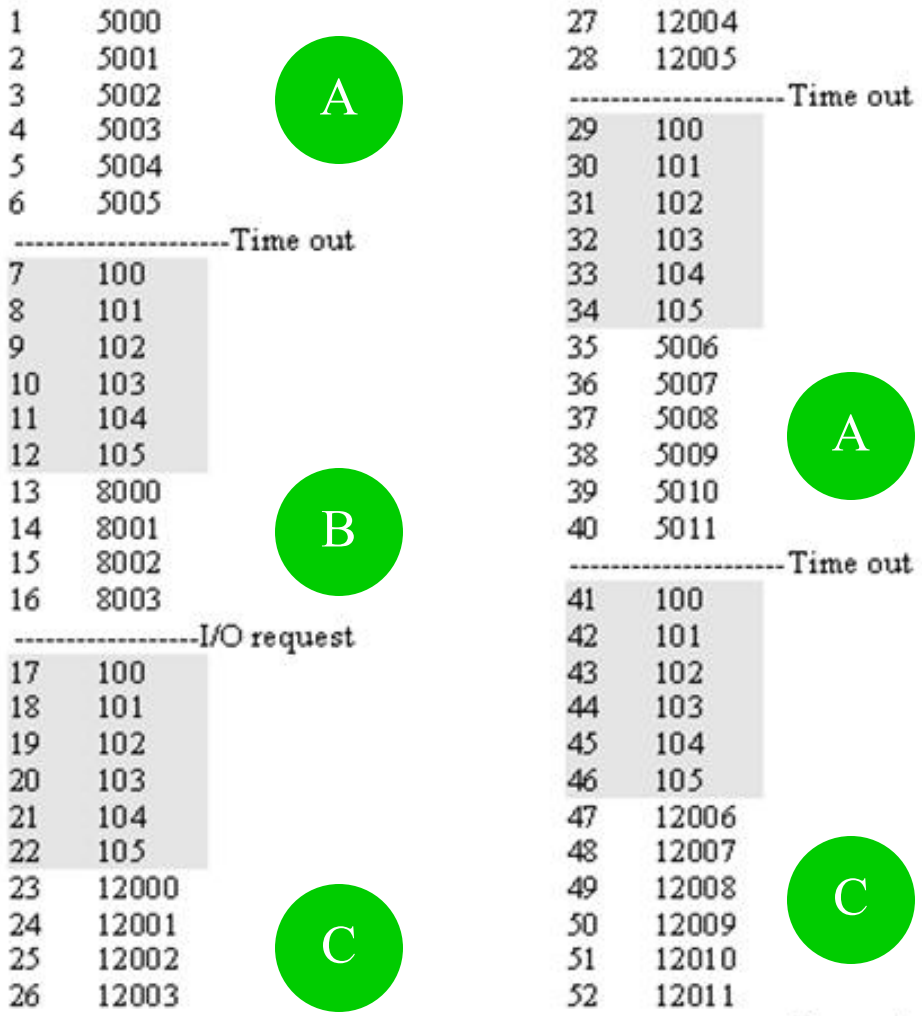


Figure 3.3 Combined Trace of Processes of Figure 3.1

Concurrent Execution of Processes (Process States)

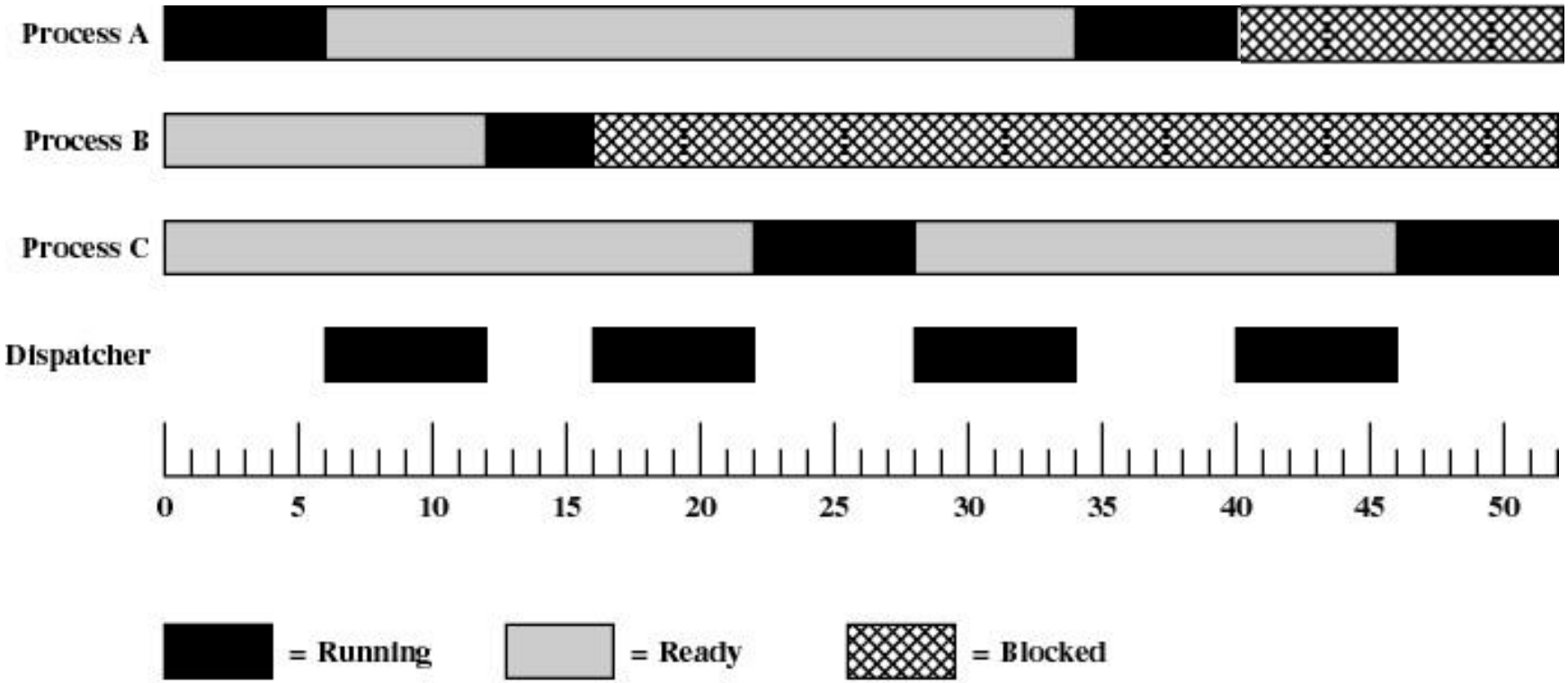


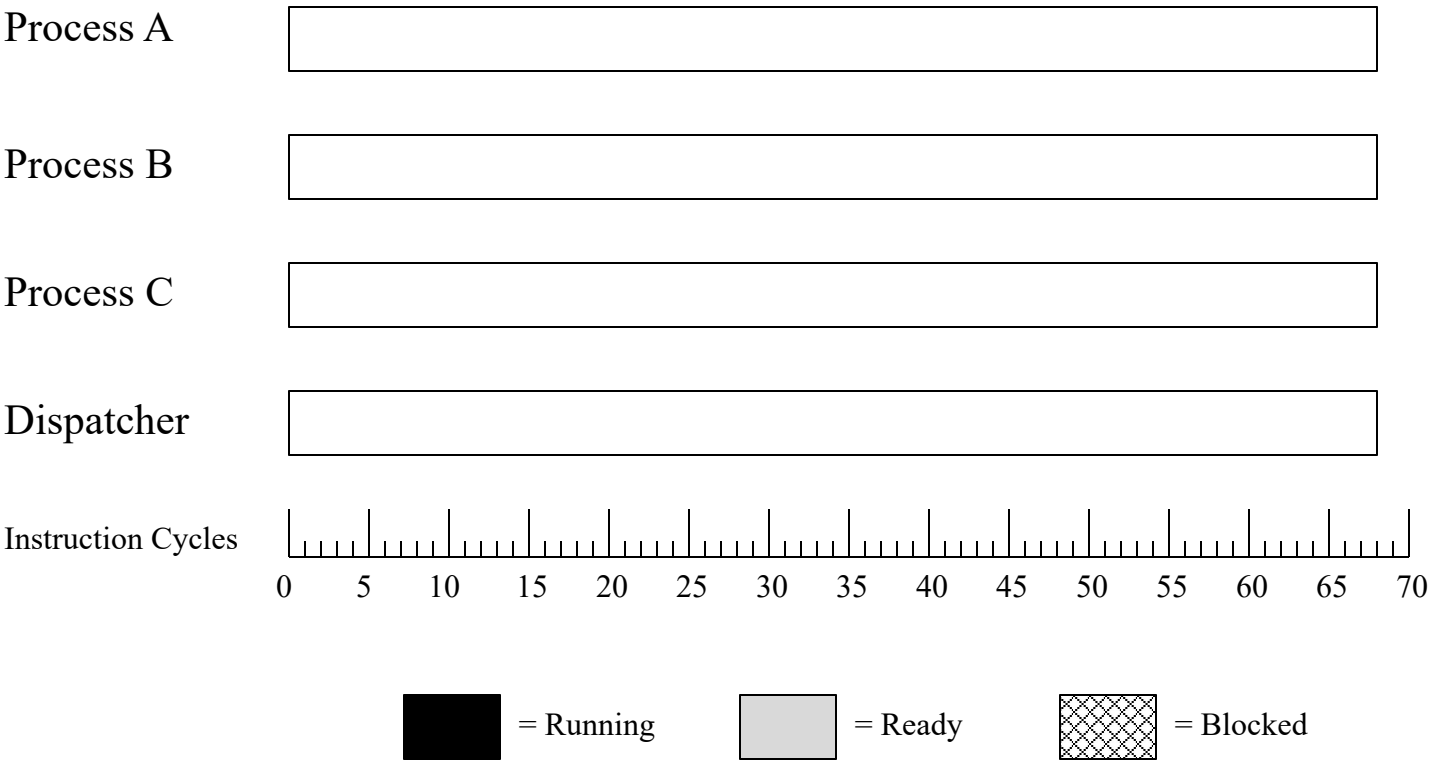
Figure 3.6 Process States for Trace of Figure 3.3

(a) The following table shows the trace addresses for 3 processes, A, B and C and the dispatcher in memory. These processes are running concurrently on a single processor system. Process A request an I/O resource at instruction cycle 4 and wait up to 30 instruction cycles. Then, process B request an I/O resource at instruction cycle 35 and wait up to 20 instruction cycles.

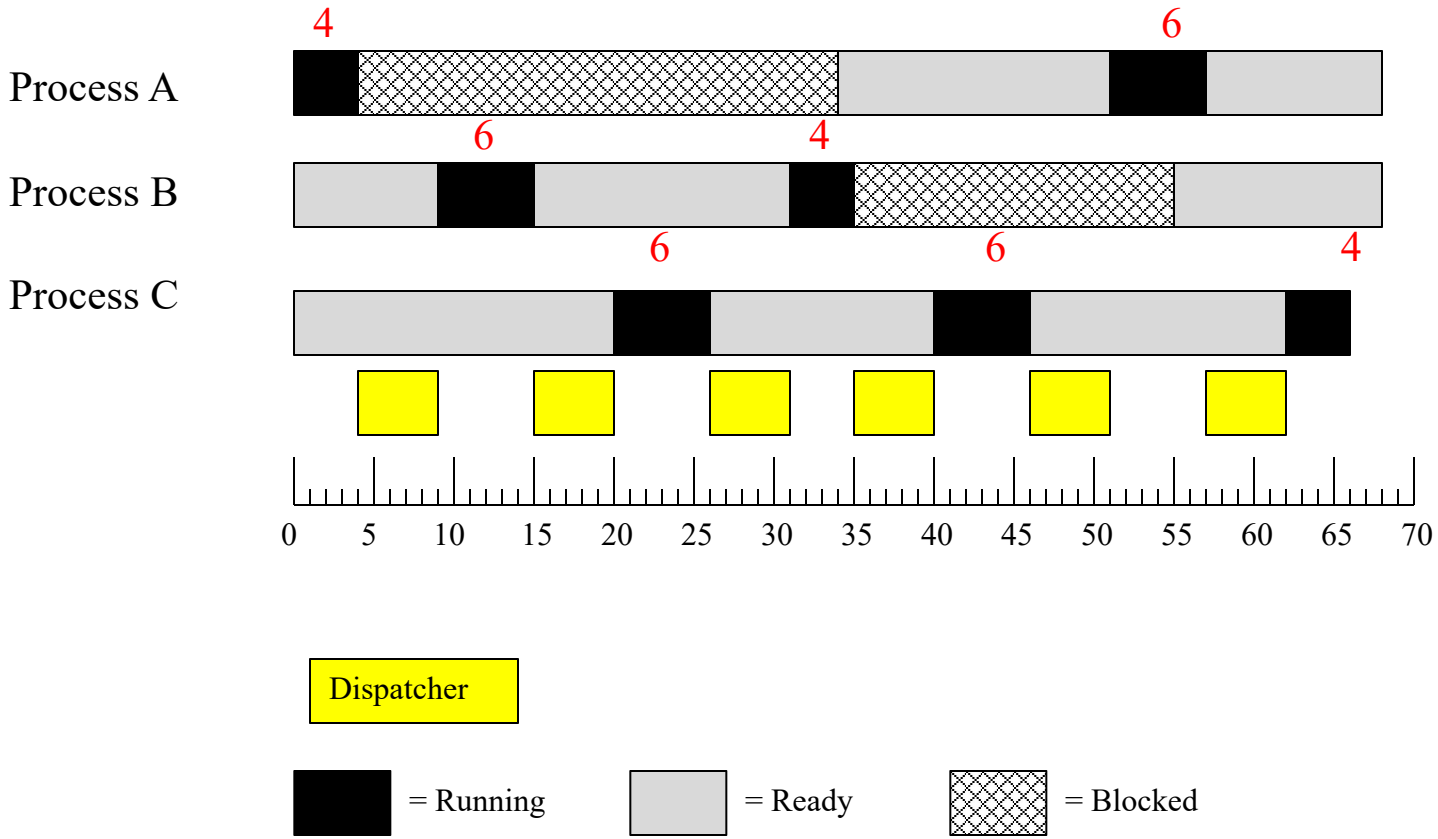
The processes and location.

Process	Addresses
A	5000 – 5020
B	7000 – 7050
C	2000 – 2015
Dispatcher	200 – 204

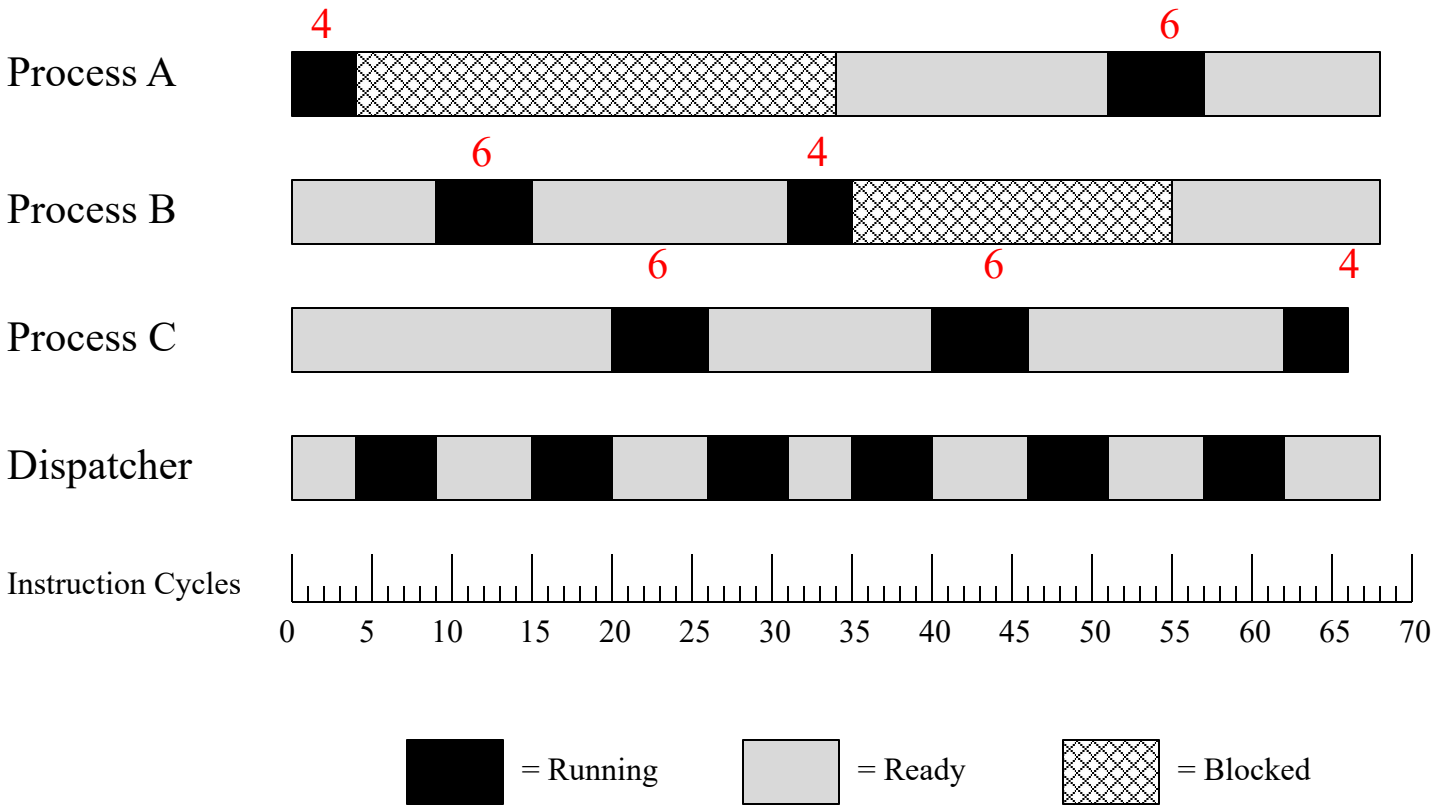
Complete the Figure 5 by drawing the process state for all processes up to 70 instruction cycles with three different indicators for *running*, *ready* and *blocked* state. Assume that the maximum CPU time for each process is 6 instruction cycles. **[6 marks]**


Process A
Process B
Process C
Dispatcher
Instruction Cycles
0 5 10 15 20 25 30 35 40 45 50 55 60 65 70
= Running
= Ready
= Blocked

Answer:

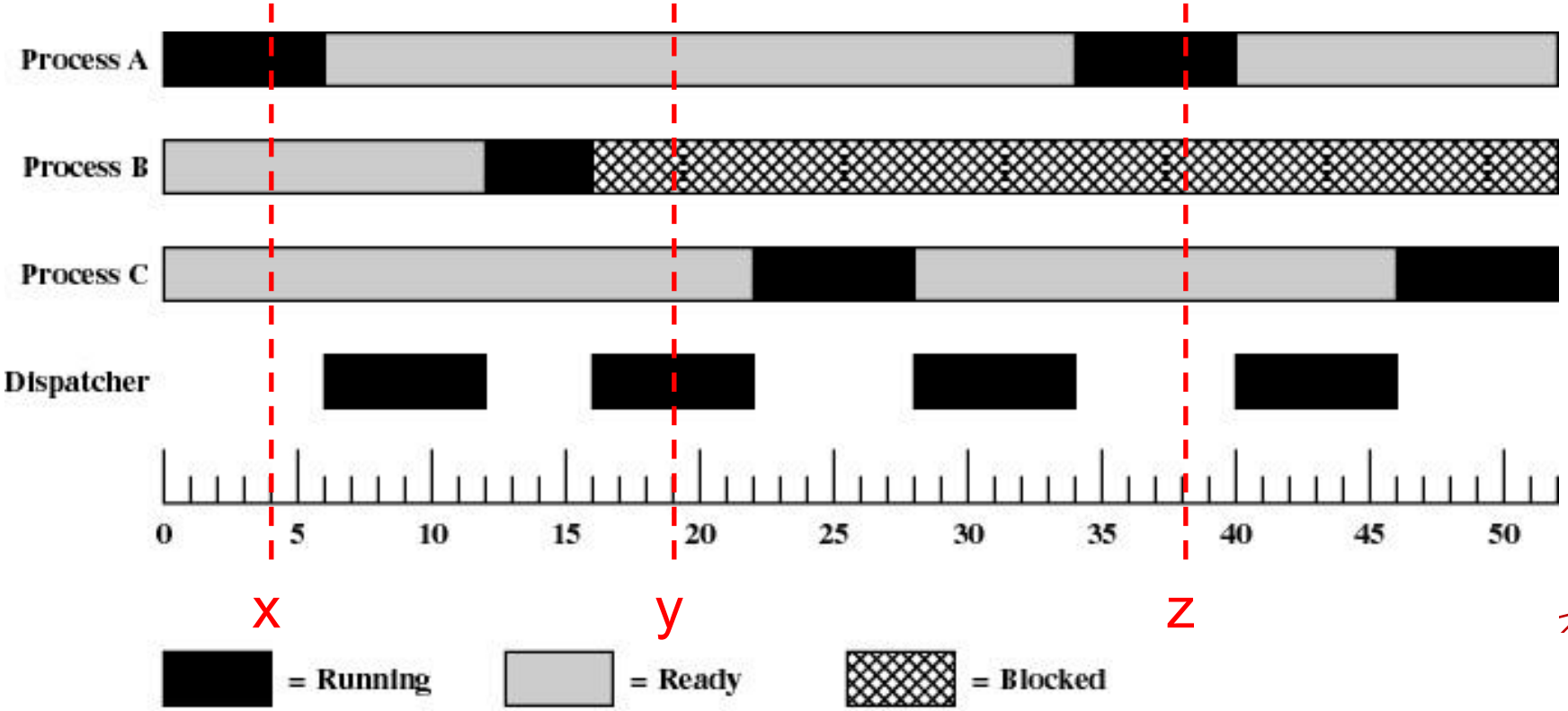

4
6
Process A
6
4
Process B
6
6
4
Process C
0 5 10 15 20 25 30 35 40 45 50 55 60 65 70
Dispatcher
= Running
= Ready
= Blocked

Answer:


4
6
Process A
6
4
Process B
6
6
4
Process C
Dispatcher
Instruction Cycles
0 5 10 15 20 25 30 35 40 45 50 55 60 65 70
= Running
= Ready
= Blocked

Exercise

What is the process state for each process at *x*, *y* and *z* ? Justify your answer.





5.2 Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – number of processes that complete their execution per time unit (e.g., 5 per second)
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – total amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)





EXERCISE (DIY!!)

The table below shows the process's location at memory for processes P, Q, R and Dispatcher. A single processor is used to run all processes. Process P request for I/O at instruction cycle 4 and wait up to 25 instruction cycles. Then, process R request for I/O at instruction cycle 23 and wait up to 24 instruction cycles. All processes run from 0 to 100 instruction cycles with three different indicators for running, ready and blocked state (unit in milliseconds). Assume that the maximum CPU burst for each process is **6 instruction cycles**. Calculate the following questions [8m]:

Process	Addresses
P	3000-3050
Q	5000-5014
R	2000-2020
Dispatcher	200-205

i. What is the Turnaround Time of process Q? [2]

ii. At time 26, what are the states for P, Q, and R? [2]

iii. At time 43, what are the states for P, Q, and R? [2]

iv. At what time does process R get the CPU after the blocked state? [2]

v. What is the Waiting Time of Process P? [2]





Optimization Criteria for Scheduling

- Max CPU utilization
  - Keep CPU busy 100% of time.
- Max throughput
  - Run as many jobs as possible in given amount of time.
- Min turnaround time
  - Move entire job in and out of system quickly.
- Min waiting time
  - Move job out of READY queue quickly.
- Min response time
  - Quickly turn around interactive requests.





5.3 Scheduling Algorithm

Based on specific policy

- Allocate CPU and move job through system

Algorithm types

- First –come, First-serve (FCFS)
- Shortest-Job-First Scheduling (SJF)
- Round-Robin Scheduling (RR)
- Priority Scheduling
- Multilevel Queue Scheduling

Current systems emphasize interactive use and response time (use preemptive policies)



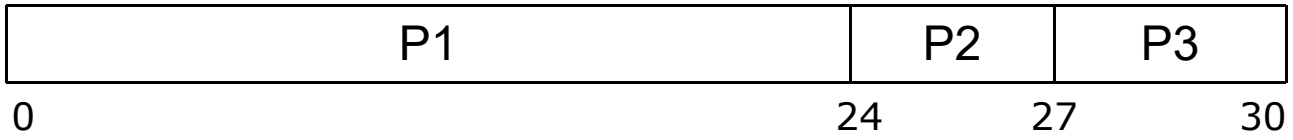


First- Come, First-Served (FCFS) Scheduling

- Non-preemptive
- Job handled based on arrival time. Earlier job arrives, earlier served.
- Consider the following three processes and their burst time.

Process	Burst Time (ms)
P1	24
P2	3
P3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
- We use Gantt Chart to illustrate a particular schedule



- Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
- Average waiting time: (0 + 24 + 27)/3 = 17



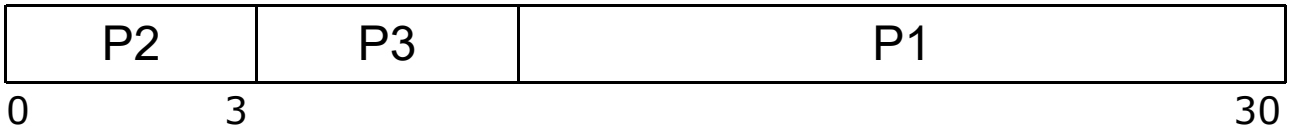


FCFS Scheduling (Cont.)

- Suppose that the processes arrive in the order:

$$P_2 , P_3 , P_1$$

- The Gantt chart for the schedule is:



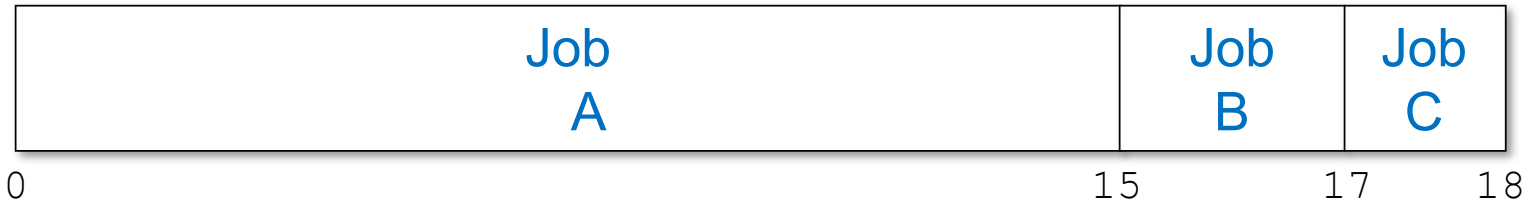
- Waiting time for $P_1 = 6; P_2 = 0; P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3
- Much better than previous case
- **Convoy effect** - short process behind long process
  - Consider one CPU-bound and many I/O-bound processes



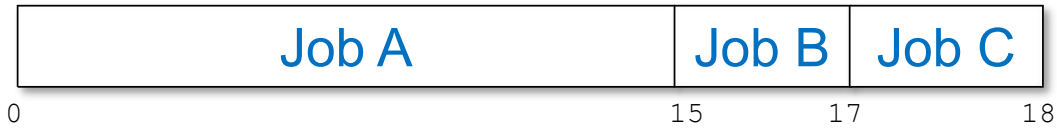
FCFS: Example 1

Three processes arrived in order Job A, Job B, Job C at time t_0 with different CPU burst. Calculate the average waiting time and average turn around time. Assume that FCFS algorithm applied.

Process	Burst Time
Job A	15
Job B	2
Job C	1



(Continued Next)



Solution:

Process	Burst Time	Waiting Time	Turnaround Time
Job A	15	0	15 – 0 = 15
Job B	2	15	17 – 0 = 17
Job C	1	17	18 – 0 = 18

- Average Waiting Time = $\frac{0+15+17}{3} = 10.67$
- Average Turnaround Time = $\frac{15+17+18}{3} = 16.67$

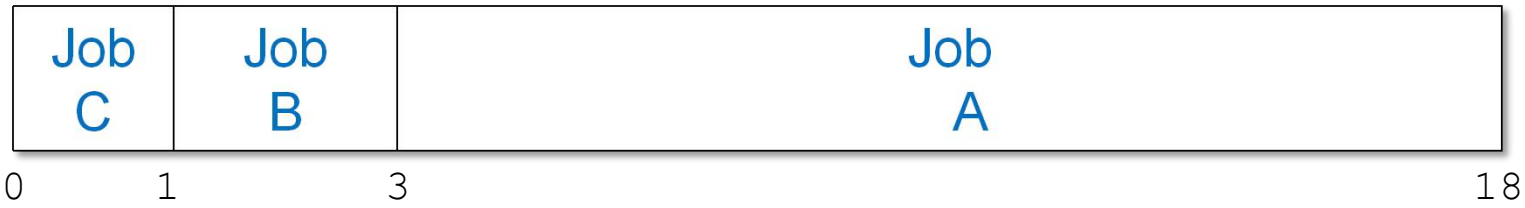
Turnaround Time = Completion time – Arrival time

Waiting Time = Turnaround time time – Burst time

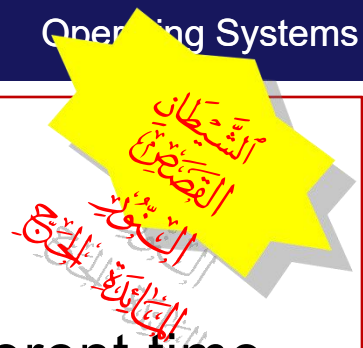
FCFS: Example 2

Three processes arrived in order Job C, Job B, Job A at time t_0 with different CPU burst time. Calculate the average waiting time and average turn around time. Assume that FCFS algorithm applied.

Process	CPU Burst Time
Job C	1
Job B	2
Job A	15



- Average Waiting Time = $\frac{0+1+3}{3} = 1.33$
- Average Turnaround Time = $\frac{1+3+18}{3} = 7.3$



Exercise 1:

Three processes Job A, Job B, Job C arrived at different time with the length of CPU burst given in milliseconds (*ms*). Calculate the average waiting time and average turn around time. Assume that FCFS algorithm applied.

Arrival Time	Process	CPU Burst
t_0	Job A	15
t_1	Job B	2
t_2	Job C	1

Turnaround Time = Completion time – Arrival time

Waiting Time = Turnaround time time – Burst time

Solution 1 :



Arrival Time	Process	CPU Burst	Waiting Time	Turnaround Time
t_0	Job A	15	0	15 – 0 = 15
t_1	Job B	2	15 – 1 = 14	17 – 1 = 16
t_2	Job C	1	17 – 2 = 15	18 – 2 = 16

- Average Waiting Time = $\frac{0+14+15}{3} = 9.67$ ms
- Average Turnaround Time = $\frac{15+16+16}{3} = 15.67$ ms

FCFS:

- Good for batch systems
- Unacceptable in interactive systems
  - Unpredictable waiting/turnaround time

- Disadvantages
  - Average waiting/turnaround time varies; seldom minimized
  - *Convoy effect* whereby short processes queue behind long processes;
    - Results in lower CPU and device utilization



Shortest-Job-First (SJF)/SJN

- None pre-emptive
- Also known as Shortest Job Next (SJN)
- Associate with each process the length of its next CPU burst/cycle time
  - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
  - How do we know what is the length of the next CPU request
  - Could ask the user
    - what if the user lies?



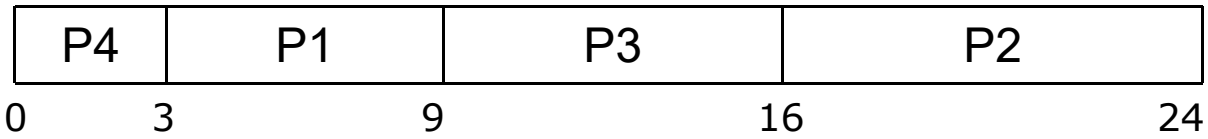


Example of SJF/SJN

- Consider the following four processes and their burst time

Process	Burst Time (ms)
P1	6
P2	8
P3	7
P4	3

- SJF scheduling chart



Turnaround Time = Completion time – Arrival time

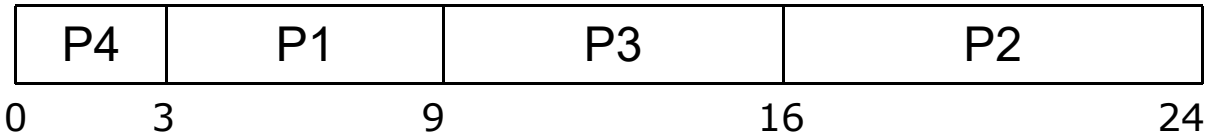
Waiting Time = Turnaround time time – Burst time



Silberschatz, Galvin and Gagne ©2013



Solutions



Process	Burst time (ms)	Turnaround time	Waiting Time
P1	6	9 – 0 = 9	9 – 6 = 3
P2	8	24 – 0 = 24	24 – 8 = 16
P3	7	16 – 0 = 16	16 – 7 = 9
P4	3	3 – 0 = 3	3 – 3 = 0
		52/4 = 13	28/4 = 7

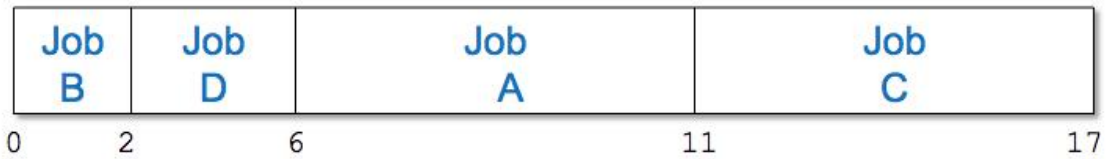


SJF/SJN: Example 3

Four processes Job A, Job B, Job C and Job D arrived at time t_0 with different CPU burst. Calculate the average waiting time and average turn around time. Assume that SJN algorithm applied.

Process	CPU Burst
Job A	5
Job B	2
Job C	6
Job D	4

Solution:



Process	Burst Time	Waiting Time	Turnaround Time
Job A	5	6	11
Job B	2	0	2
Job C	6	11	17
Job D	4	2	6

- Average Waiting Time :

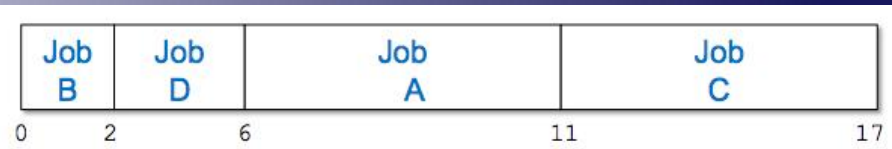
  $= \frac{A + B + C + D}{4} = \frac{6 + 0 + 11 + 2}{4} = 4.75$

- Average turnaround time:

  $= \frac{A + B + C + D}{4} = \frac{11 + 2 + 17 + 6}{4} = 9.0$



- We can see:
  - Job B finishes in its given time, (2)
  - Job D finishes in its given time plus the time waited for Job B to run, (4+2)
  - Job A, time given plus B's time, (5+4+2)
  - Job C, time given plus A's time, (6+ 5+4+2)



Waited time

$$\frac{(2) + (4 + 2) + (5 + 4 + 2) + (6 + 5 + 4 + 2)}{4} = 9.0$$

- As we can see:
  - The time 1st job (B) appears 4 times; The 2nd job (D) appears 3 times ; The 3rd job (A) appears 2 times and the 4th job (C) appears only once (1).

- Equation can be rewrite as:

$$\frac{(B)*4 + (D)*3 + (A)*2 + (C)*1}{4} = 9.0$$

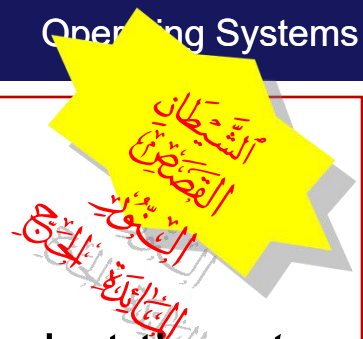
- Formula for average:

$$\frac{t_1(n) + t_2(n-1) + t_3(n-2) + \ldots + t_n(n(1))}{n}$$

n = number of job in queue; t_j $(j=1,2,3,\ldots,n)$

SJN / SJF:

- Easy implementation in batch environment
  - CPU time requirement known in advance
- Does not work well in interactive systems
- Optimal algorithm
  - All jobs are available at same time.
  - CPU estimation are available and more accurate.
- If 2 processes have the same CPU bursts (cycle time), FCFS may be used.

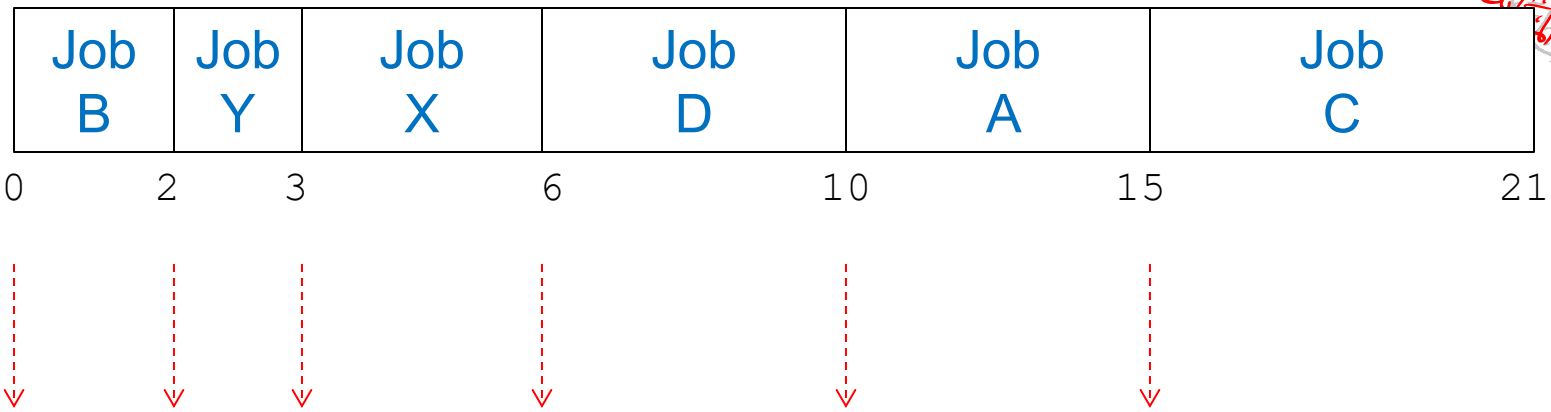


Exercise 2:

Four processes Job A, Job B, Job C and Job D arrived at time t_0 with different CPU burst in milliseconds (ms). At t_2 , another processes Job X and Job Y arrived with its own CPU burst time. Calculate the average waiting time and average turnaround time. Assume that SJN algorithm applied and each $t_i = i$ CPU Burst.

Arrival Time (t)	Process	CPU Burst
0	Job A	5
	Job B	2
	Job C	6
	Job D	4
2	Job X	3
	Job Y	1

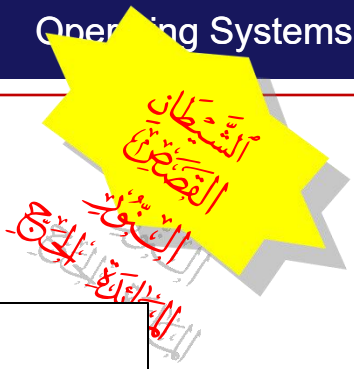
Solution 2:

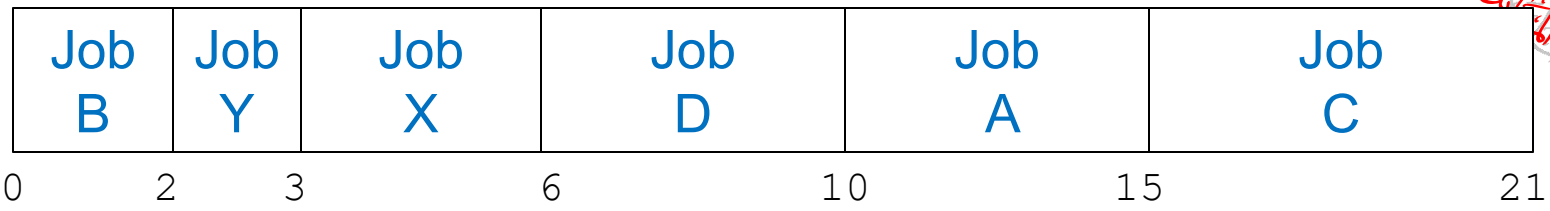


A_5	A_5	A_5	A_5	A_5	C_6
B_2	C_6	C_6	C_6	C_6	
C_6	D_4	D_4	D_4		
D_4	X_3	X_3			
	Y_1				

Processes in system to use CPU

Note: Job_{CPU}





Arrival Time	Process	CPU Burst	Waiting Time	Turnaround Time
t_0	Job A	5	10	15
	Job B	2	0	2
	Job C	6	15	21
	Job D	4	6	10
t_2	Job X	3	3 – 2 = 1	6 – 2 = 4
	Job Y	1	2 – 2 = 0	3 – 2 = 1

- Average Waiting Time = $\frac{10+0+15+6+1+0}{6} = 5.33\ ms$
- Average Turnaround Time = $\frac{15+2+21+10+4+1}{6} = 8.83\ ms$



Shortest-remaining-time-first (SRT/SRTF)

- A preemptive version of SJF is called **shortest-remaining-time-first**
- Processor allocated to job closest to completion.
  - Preemptive if the newer job has shorter completion time.
- Example illustrating the concepts of varying arrival times and preemption.

Process	Arrival Time (t)	CPU Burst	Turnaround	Waiting
P1	0	8	17 – 0 = 17	17 – 8 = 9
P2	1	4	5 – 1 = 4	4 – 4 = 0
P3	2	9	25 – 2 = 23	23 – 9 = 14
P4	3	5	10 – 3 = 7	7 – 5 = 2

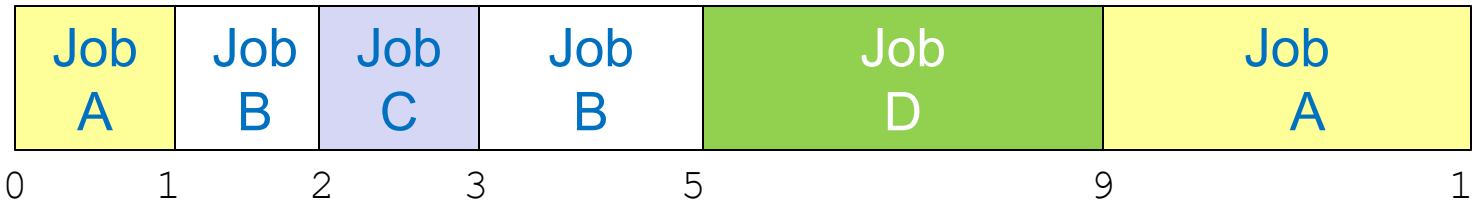


- Average turnaround time = 51 /4 = 12.75 ms
- Average waiting time = 6.25 ms



- **SRT : Example 4**

Process	Burst Time	Arrival Time
Job A	6	$t_0 = 0.0$
Job B	3	$t_1 = 1.0$
Job C	1	$t_2 = 2.0$
Job D	4	$t_3 = 3.0$



Timeline for job sequence A, B, C, D using the preemptive SRT algorithm.
Each job is interrupted after one CPU cycle if another job is waiting with less CPU time remaining.

Process	Burst Time	Arrival Time
Job A	6	0.0
Job B	3	1.0
Job C	1	2.0
Job D	4	3.0

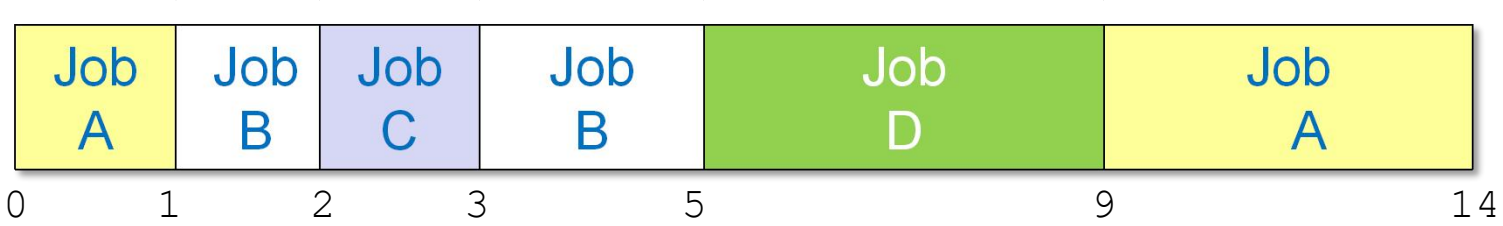
Here Job A is preempted by Job B because Job B has less CPU time remaining.

Here Job B is preempted by Job C because Job C has less CPU time remaining.

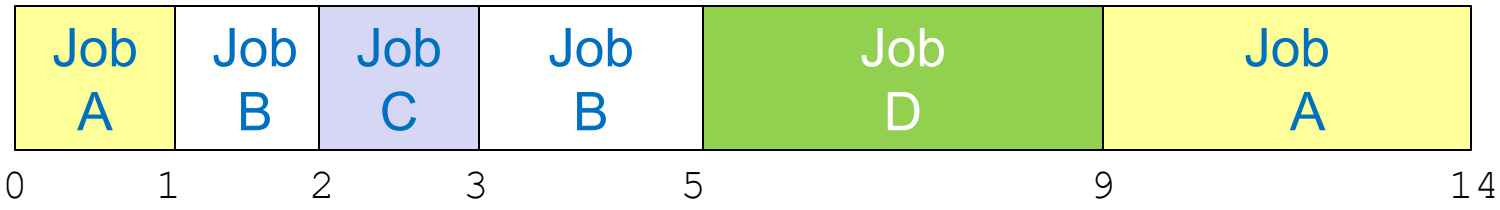
Now Job B can resume because Job C has finished.

Job D run next because it needs less CPU time to finish than does Job A.

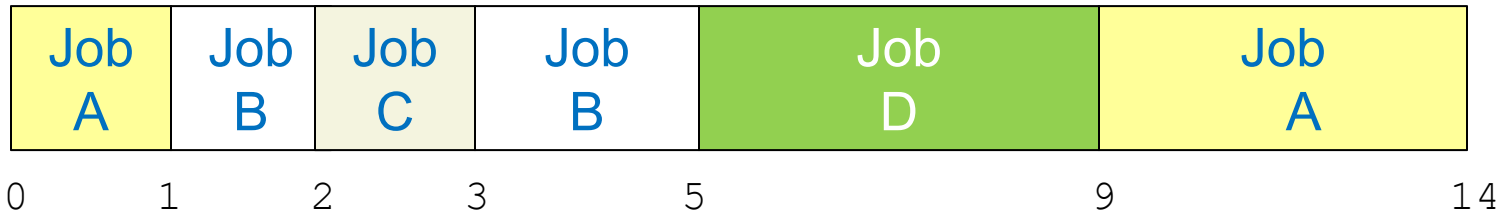
Here Job A is finally allow to finish (resume).



Process	Burst Time	Arrival Time	Completion Time
Job A	6	0.0	14
Job B	3	1.0	5
Job C	1	2.0	3
Job D	4	3.0	9

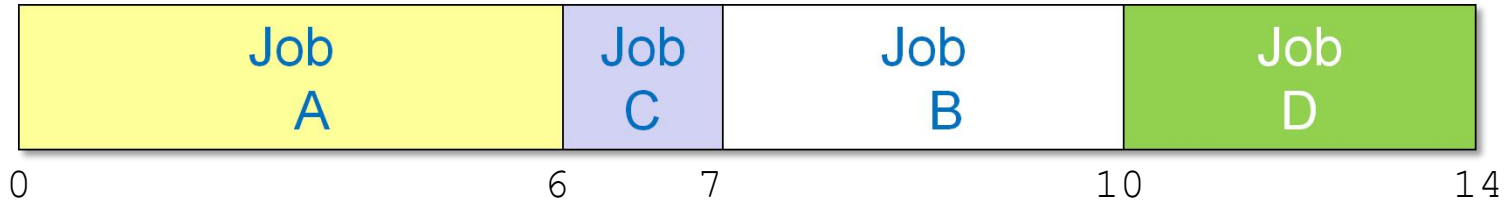


Process	Burst Time	Arrival Time	Turnaround time	Waiting time
Job A	6	0.0	14 – 0 = 14	14 – 6 = 8
Job B	3	1.0	5 – 1 = 4	4 – 3 = 1
Job C	1	2.0	3 – 2 = 1	1 – 1 = 0
Job D	4	3.0	9 – 3 = 6	6 – 4 = 2



- **Average turnaround time =** (14+4+1+6)/4 = 6.25 ms
- **Average waiting time =** (8+1+0+2)/4 = 2.75 ms

- **Example 5 (Nonpreemptive SJN/SJF)**

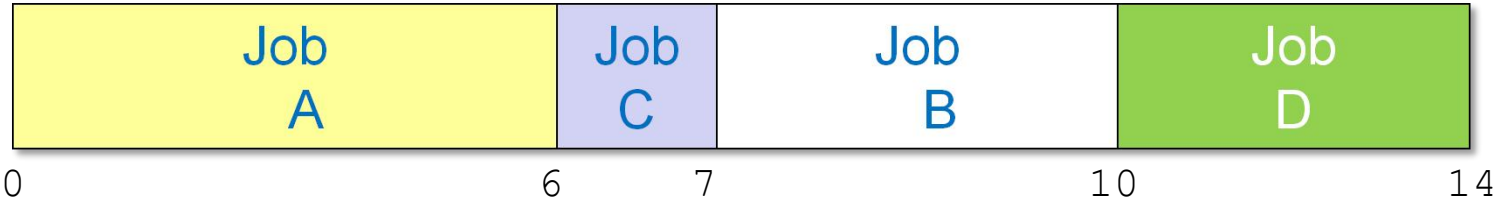


Timeline for the same job sequence A, B, C, D using the nonpreemptive SJN algorithm.

Process	CPU Burst	Arrival Time	Completion Time
Job A	6	0.0	6
Job B	3	1.0	10
Job C	1	2.0	7
Job D	4	3.0	14

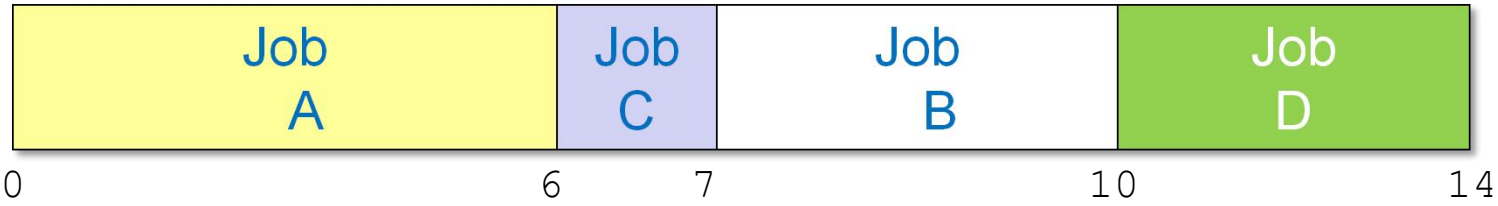
Note: **Job A** initially in the READY queue so it runs first and continues with the rest.

(Continued Next)



Process	CPU Burst	Time	Job in CPU	Job in READY queue
Job A	6	0.0	A	-
Job B	3	1.0	A	B
Job C	1	2.0	A	B, C
Job D	4	3.0	A	B, C, D
		6.0	C	B, D
		7.0	B	D
		10.0	D	-

(Continued Next)



- Turnaround time:

Job	**:**	**A**	**B**	**C**	**D**
Turnaround	:	6	9	5	11

- Average turnaround time: $\frac{6 + 9 + 5 + 11}{4} = 7.75$

- Comparison between example 4 and 5:
  - SRT (6.25) is faster than SJN/SJF (7.75)
    → (0.12% faster)

SRT:

- Often used in batch environments
  - Short jobs given priority
- Cannot implement in interactive system
  - Requires advance CPU time knowledge
- Involves more overhead than SJN/SJF
  - System monitors CPU time for `READY` queue jobs
  - Performs context switching



Round Robin (RR)

- Preemptive. Used extensively in interactive systems.
- Each process gets a small unit of CPU time (**time quantum** q). After this time has elapsed, the process is preempted and added to the end of the ready queue (FCFS).
- Time quantum size
  - Crucial to system performance
  - Varies from 100 ms to 1-2 seconds
- If there are N processes in the ready queue and the time quantum is q, then each process gets $1/N$ of the CPU time in chunks of at most q time units at once. No process waits more than $(N-1)q$ time units.
- Timer interrupts every quantum to schedule next process.



- If job CPU burst > time quantum
  - Job preempted and placed at end of READY queue
  - Information saved in PCB

- If job CPU burst < time quantum
  - 1) Job finished:
    - allocated resources released and job returned to user
  - 2) Interrupted by I/O request:
    - information saved in PCB and linked to I/O queue
    - Once I/O request satisfied, Job returns to end of READY queue and awaits CPU

(Continued Next)

- Efficiency
  - Depends on time quantum size
    - In relation to average CPU burst



- Quantum too **large** (larger than most CPU bursts)
  - Algorithm reduces to FCFS scheme

- Quantum too **small**
  - Context switching occurs
    - Job execution slows down
    - Overhead dramatically increased



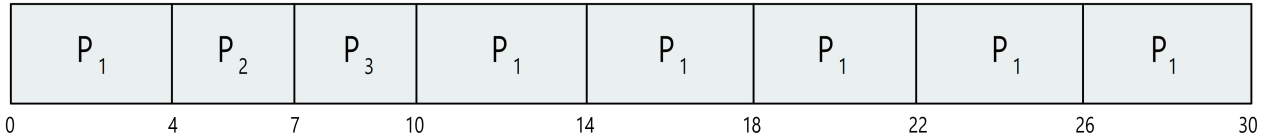


Example of RR with Time Quantum = 4

- Consider the following three processes and their burst time

Process	Burst Time
P_1	24
P_2	3
P_3	3

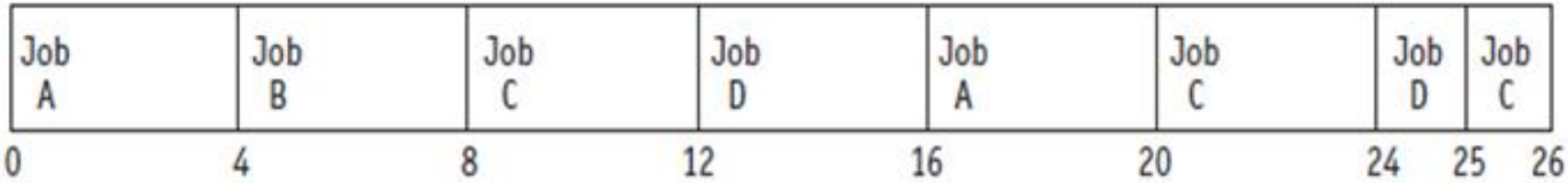
- The Gantt chart is:



- The average waiting time under the RR policy is often longer.
- Typically, higher average turnaround than SJF, but better ***response.***
- q should be large compared to context switch time.
- q is usually 10ms to 100ms, context switch < 10 usec



- **RR : Example 6**



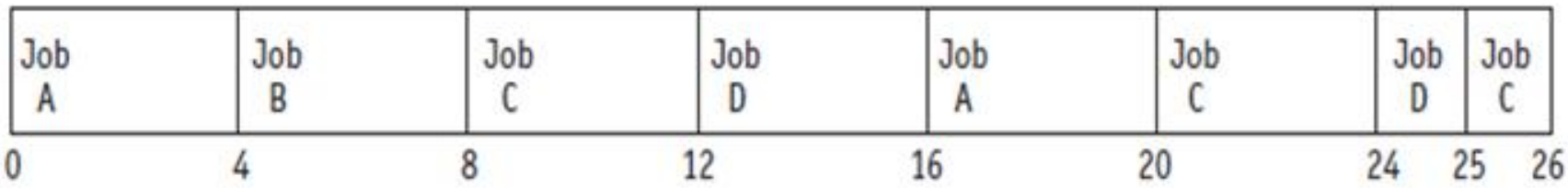
Timeline for job sequence A, B, C, D using the preemptive round robin algorithm with time slices of 4 ms.

Process	CPU Burst	Arrival Time
Job A	8	0.0
Job B	4	1.0
Job C	9	2.0
Job D	5	3.0

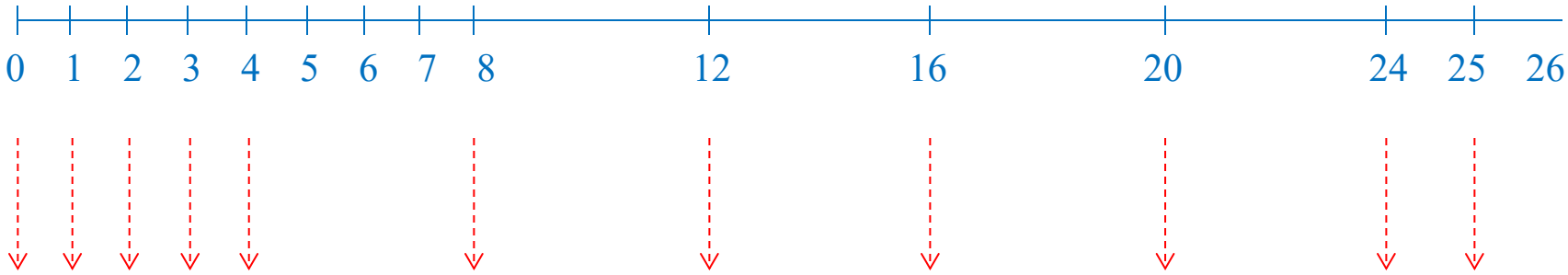


Time Slice =
4 milliseconds

(Continued Next)



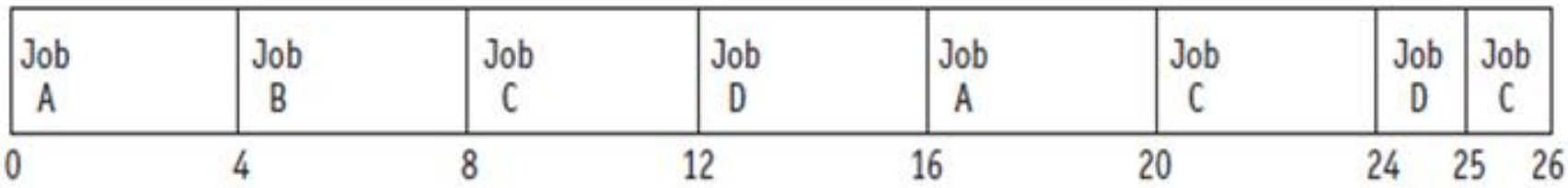
Time:



A_8 A_7 A_6 A_5 A_4	A_4	A_4	A_4			
B_4 B_4 B_4 B_4	C_9	C_5	C_5	C_5	C_1	C_1
C_9 C_9 C_9	D_5	D_5	D_1	D_1	D_1	
D_5 D_5						

Note: Job_{CPU}

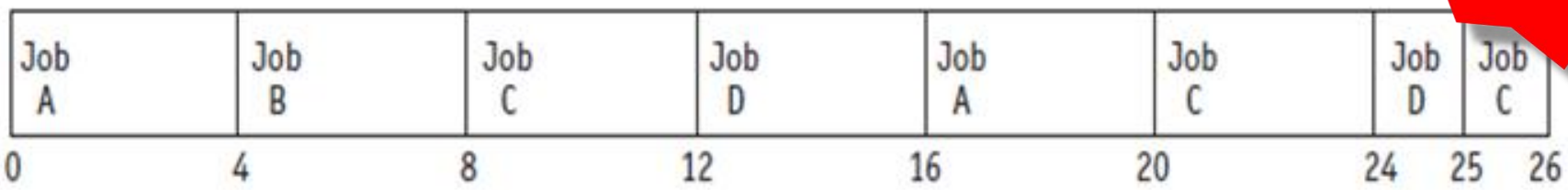
Time Slice = 4 ms



Process	CPU Burst	Arrival Time	Completion Time	Turnaround Time
Job A	8	0.0	20	20 – 0 = 20
Job B	4	1.0	8	8 – 1 = 7
Job C	9	2.0	26	26 – 2 = 24
Job D	5	3.0	25	25 – 3 = 22

Average Turnaround Time = $\frac{20+7+24+22}{4} = \frac{73}{4} = 18.25$

(Continued Next)



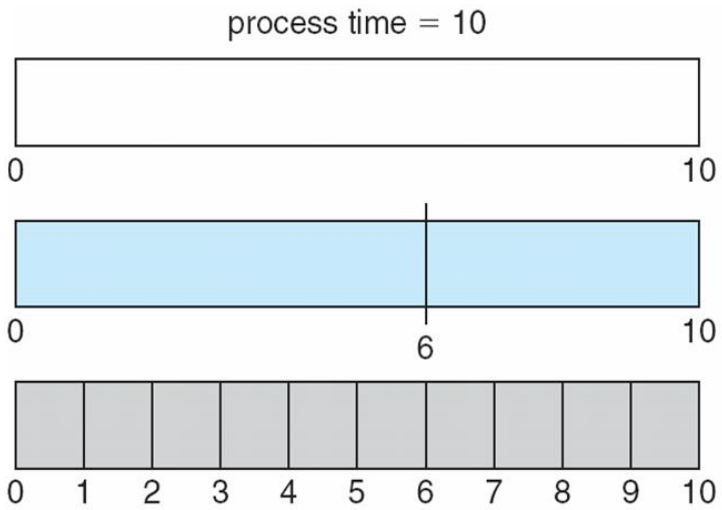
Process	CPU Burst	Arrival Time	Waiting Time
Job A	8	0.0	(0 – 0) + (16 – 4) = 12
Job B	4	1.0	4 – 1 = 3
Job C	9	2.0	(8 – 2) + (20 – 12) + (25 – 24) = 15
Job D	5	3.0	(12 – 3) + (24 – 16) = 17

Average Waiting Time = $\frac{12+3+15+17}{4} = \frac{47}{4} = 11.75$



Time Quantum and Context Switch Time

- The performance of the RR algorithm depends on the size of the time quantum. If the time quantum is extremely small (say, 1 millisecond), RR can result in a large number of context switches.



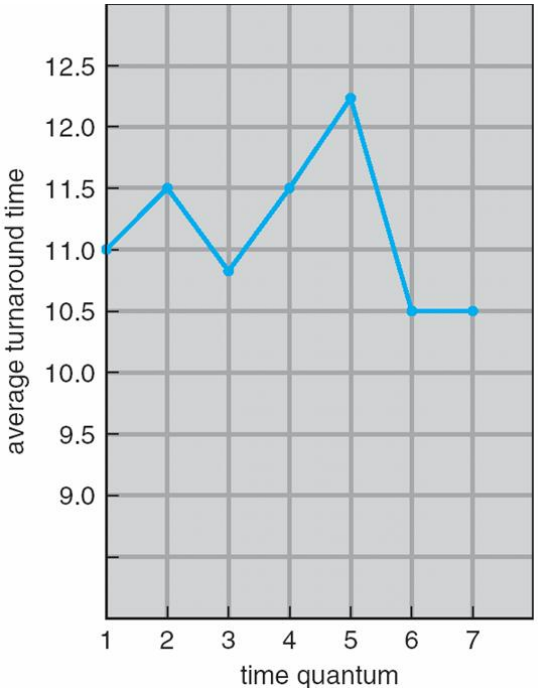
quantum	context switches
12	0
6	1
1	9





Turnaround Time Varies With The Time Quantum

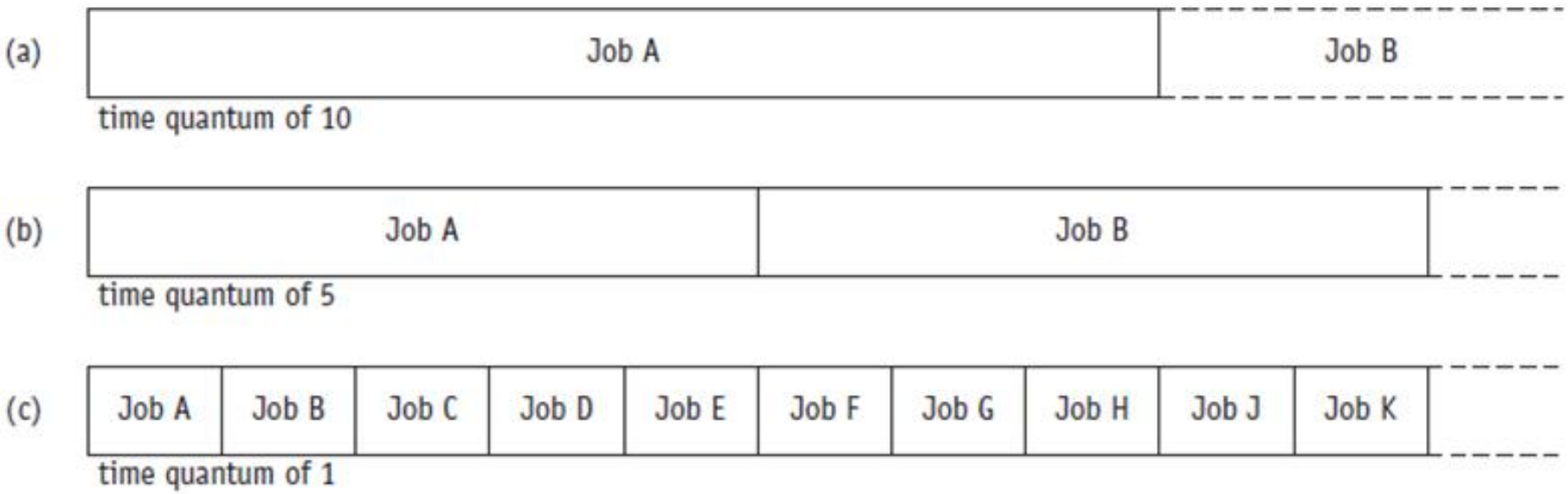
- The average turnaround time of a set of processes does not necessarily improve as the time-quantum size increases. In general, the average turnaround time can be improved if most processes finish their next CPU burst in a single time quantum.



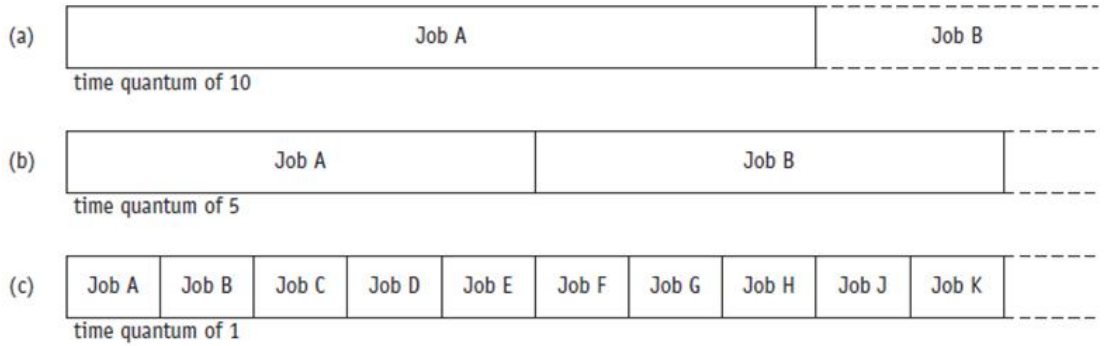
process	time
P_1	6
P_2	3
P_3	1
P_4	7



- **RR : Example 7**



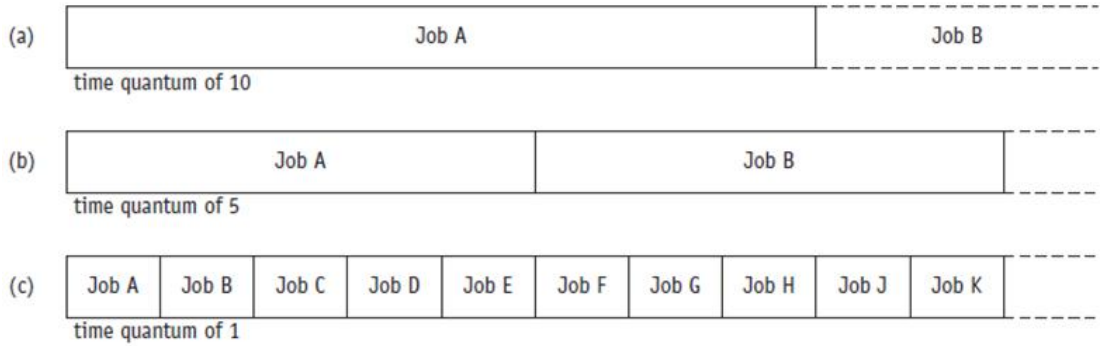
Context switches for three different time quantums. In (a), Job A (which requires only 8 cycles to run to completion) finishes before the time quantum of 10 expires. In (b) and (c), the time quantum expires first, interrupting the jobs.



- Case (a)
  - There is no context switch
  - Job A runs to completion since CPU burst end shortly before time quantum expires
  - No different between RR and FCFS policies

- Case (b)
  - There is 1 context switch for the rest of Job A.
  - Job A is preempted once when the time quantum expires

(Continued Next)



- Case (c)
  - There is 10 context switches for 2nd cycle of Job A.
  - The job is preempted every time the time quantum expires

- Weakness for case (b) & (c):
  - Overhead become costly
  - Turnaround time suffers

(Continued Next)



Priority Scheduling

- A priority number (integer) is associated with each process.
- The CPU is allocated to the process with the highest priority. (smallest integer ≡ highest priority)
  - Preemptive
  - Nonpreemptive
- SJF is simply of priority scheduling where it uses the inverse of the next CPU burst time as its priority –
  - The smaller the CPU burst, the higher the priority.
- If two jobs having the same priority are READY, it works on a FIRST COME, FIRST SERVED basis.
- Problem ≡ **Starvation** – low priority processes may never execute.
- Solution ≡ **Aging** – as time progresses increase the priority of the process.





Example of Priority Scheduling

- Consider the following five processes and their burst time

Process	CPU Burst	Priority	Turnaround time	Waiting time
P1	10	3	16 – 0 = 16	16 – 10 = 6
P2	1	1	1 – 0 = 1	1 – 1 = 0
P3	2	4	18 – 0 = 18	18 – 2 = 16
P4	1	5	19 – 0 = 19	19 – 1 = 18
P5	5	2	6 – 0 = 6	6 – 5 = 1

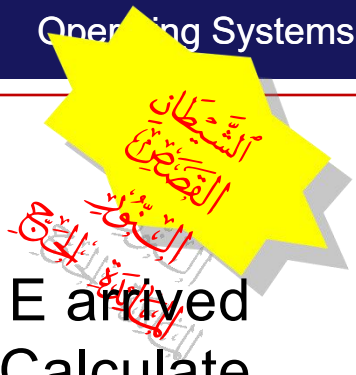
- Priority scheduling Gantt Chart



- Average waiting time = 8.2 msec



- System Administrator or Processor Manager use different methods of assigning priorities.
- Processor Manager priority assignment methods:
  - *Memory requirements*
    - Jobs requiring large amounts of memory
    - Allocated lower priorities (vice versa)
  - *Number and type of peripheral devices*
    - Jobs requiring many peripheral devices
    - Allocated lower priorities (vice versa)
  - *Total CPU time*
    - Jobs having a long CPU burst
    - Given lower priorities (vice versa)
  - *Amount of time already spent in the system (aging)*
    - Total time elapsed since job accepted for processing
    - Increase priority if job in system unusually long time



Exercise 3:

Five processes Job A, Job B, Job C, Job D and Job E arrived at time t_0 with different CPU burst and priority level. Calculate the average waiting time and average turnaround time. Assume the following scheduling algorithm applied separately.

a) FCFS, SJN and Priority
b) Which one is the best in term of average waiting time and turnaround time? Why?

Process	CPU Burst	Priority
Job A	10	3
Job B	1	1
Job C	2	4
Job D	1	5
Job E	5	2



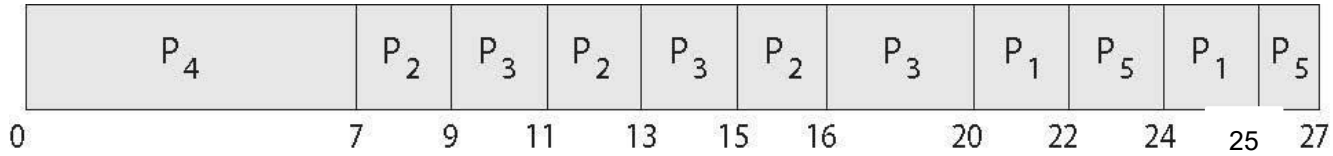
Combining Priority Scheduling and RR

- System executes the highest priority process; processes with the same priority will be run using round-robin (q = 2).
- Consider the following five processes and their burst time

• identify the priority
• if 2 processes has same priority, apply FCFS

Process	Burst Time	Priority
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

Priority scheduling Gantt Chart

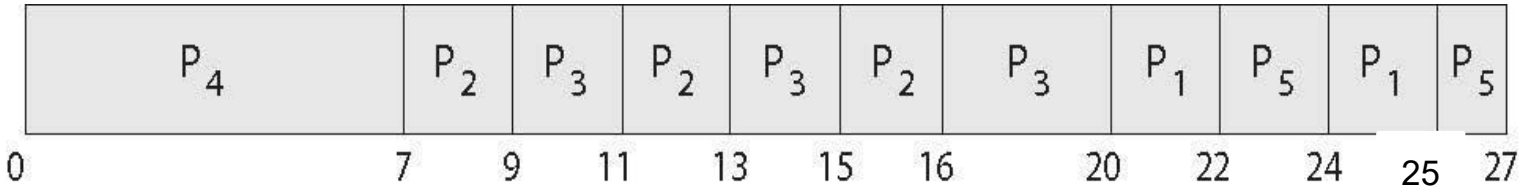


- Average waiting time = 13.6 ms (previously 8.2 ms)





Combining Priority Scheduling and RR



Process	Burst Time	Priority	Completion time	Turnaround time	Waiting time
P_1	4	3	25	25 – 0 = 25	25 – 4 = 21
P_2	5	2	16	16 – 0 = 16	16 – 5 = 11
P_3	8	2	20	20 – 0 = 20	20 – 8 = 12
P_4	7	1	7	7 – 0 = 7	7 – 7 = 0
P_5	3	3	27	27 – 0 = 27	27 – 3 = 24

- **Average waiting time = 13.6 ms**
- **Average turnaround time = 19 ms**

Turnaround Time = Completion time – Arrival time

Waiting Time = Turnaround time time – Burst time



Silberschatz, Galvin and Gagne ©2013



Multilevel Queue

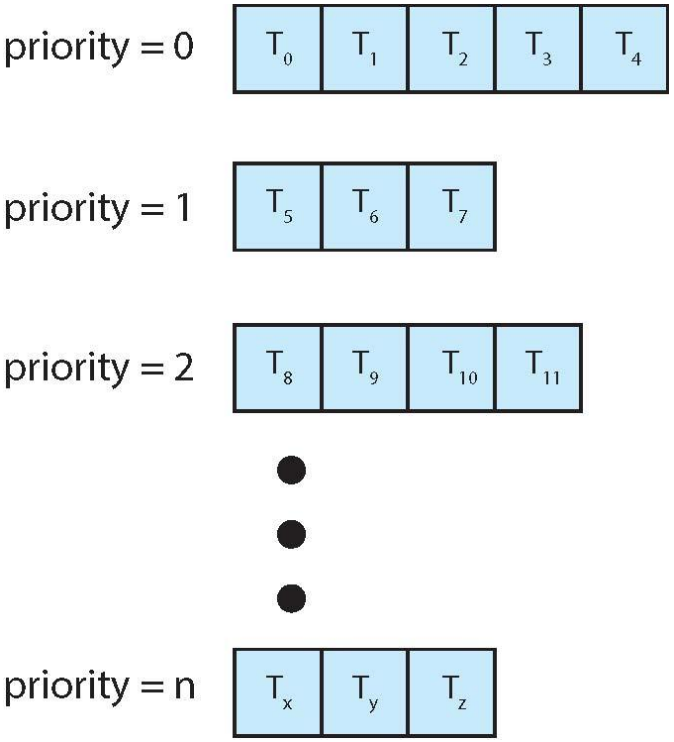
- Hybrid environment. Ready queue is partitioned into separate queues, eg:
  - **foreground** (interactive)
  - **background** (batch)
- Process permanently in a given queue
- Each queue has its own scheduling algorithm:
  - foreground – RR
  - background – FCFS
- Scheduling must be done between the queues:
  - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes;
  - i.e., 80% to foreground in RR, 20% to background in FCFS





Separate Queue For Each Priority

- Works in conjunction with several other schemes
- Works well in systems with jobs grouped by common characteristics.





- Four primary methods:

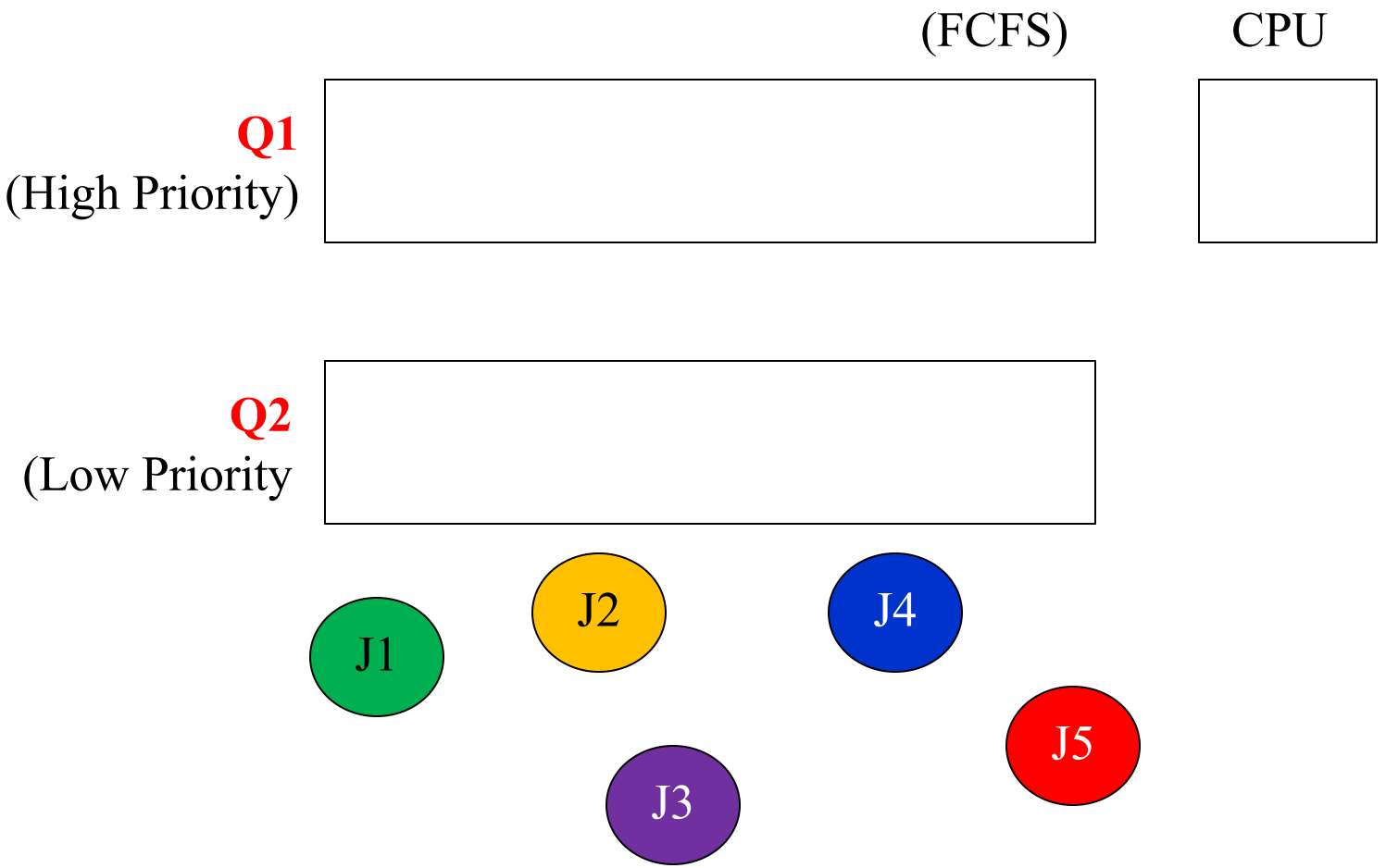
1. Not allowing movement between queues
2. Movement jobs between queues
3. Variable Time Quantum Per Queue
4. Aging- giving special treatment to jobs that have been in the system for a long time

(Continued Next)

Case 1: No Movement Between Queues

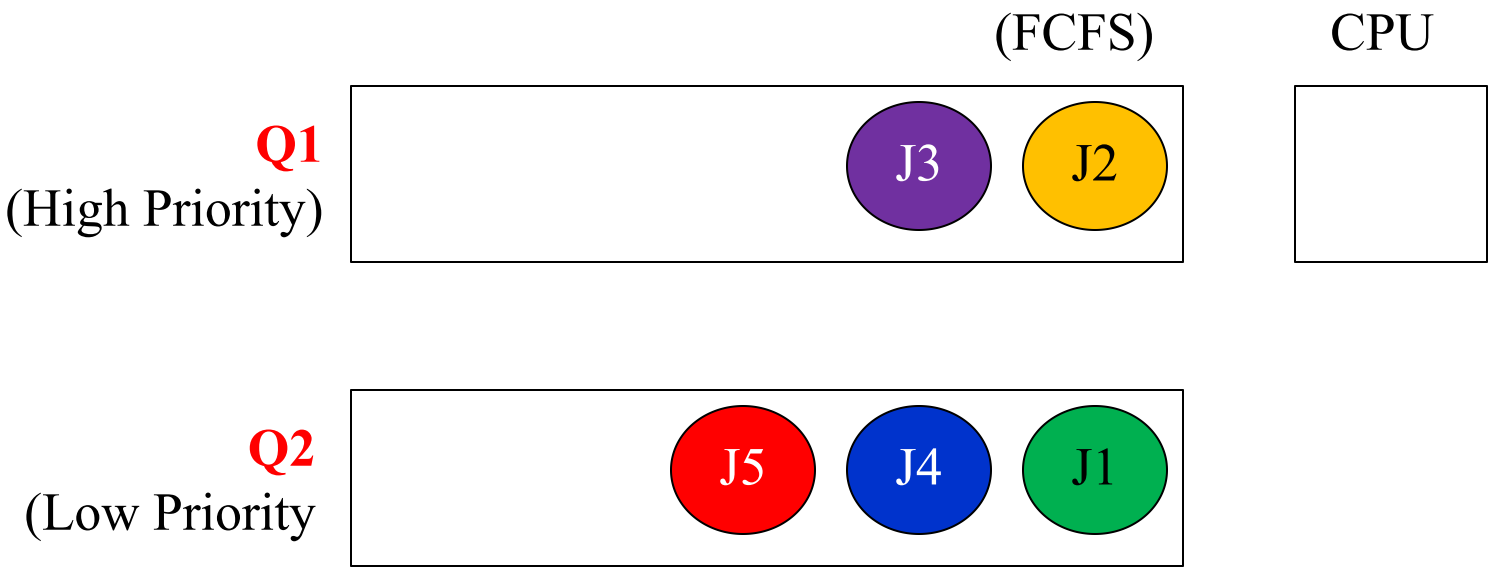
- Simple
- Rewards high-priority jobs
  - Processor allocated using FCFS
- Processor allocated to lower-priority jobs
  - Only when high-priority queues empty
- Good environment
  - Few high-priority jobs
  - Spend more time with low-priority jobs

Case 1: No Movement Between Queues



Arrival of jobs: J1, J2, J3, J4, J5

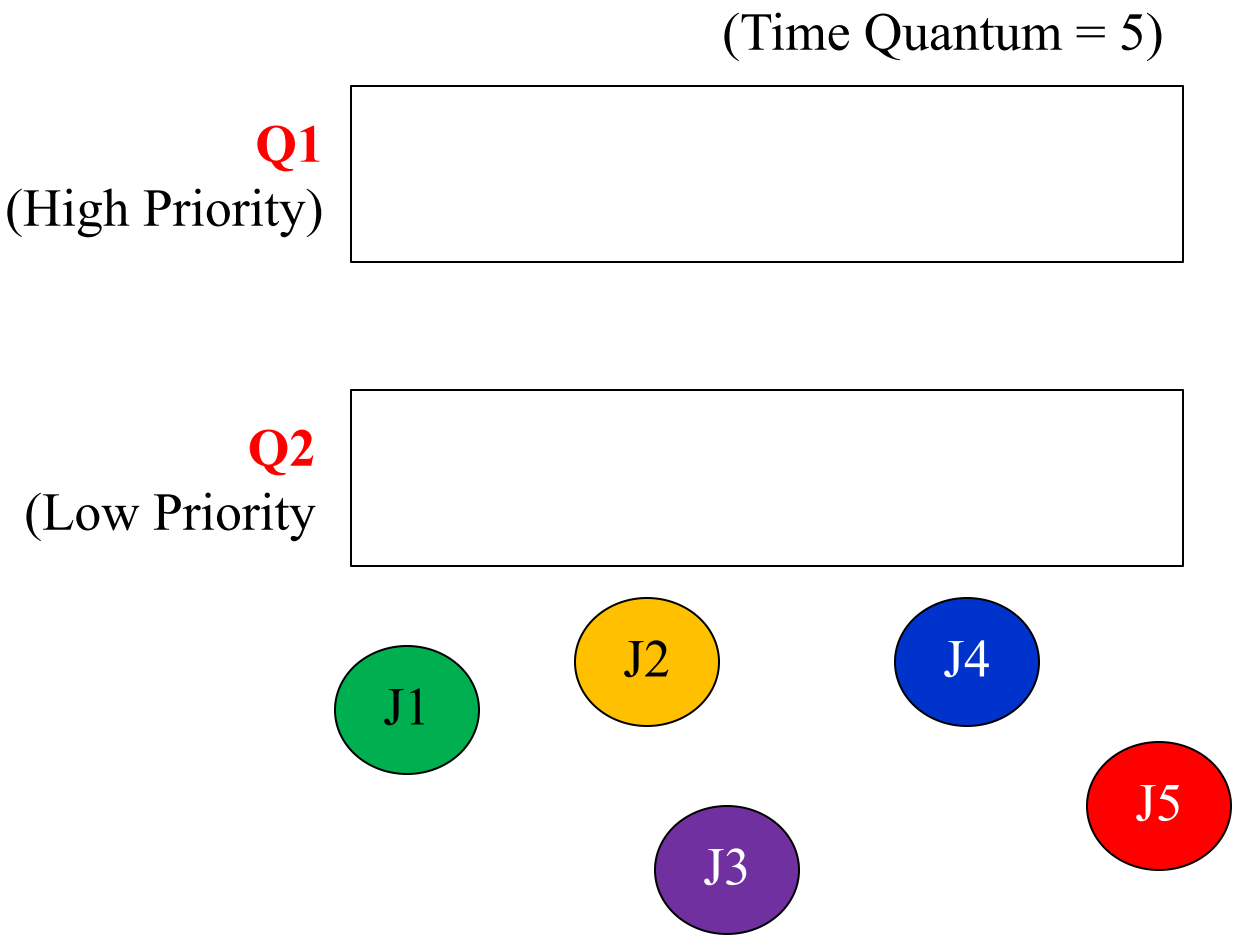
Case 1: No Movement Between Queues



Case 2: Movement Between Queues

- Processor adjusts priorities assigned to each job
- High-priority jobs
  - Initial priority favorable
    - Treated like all other jobs afterwards
- Quantum interrupt
  - Job preempted
    - Moved to next lower queue
    - May have priority increased
- Good environment
  - Jobs handled by cycle characteristics (CPU or I/O)
  - Interactive systems

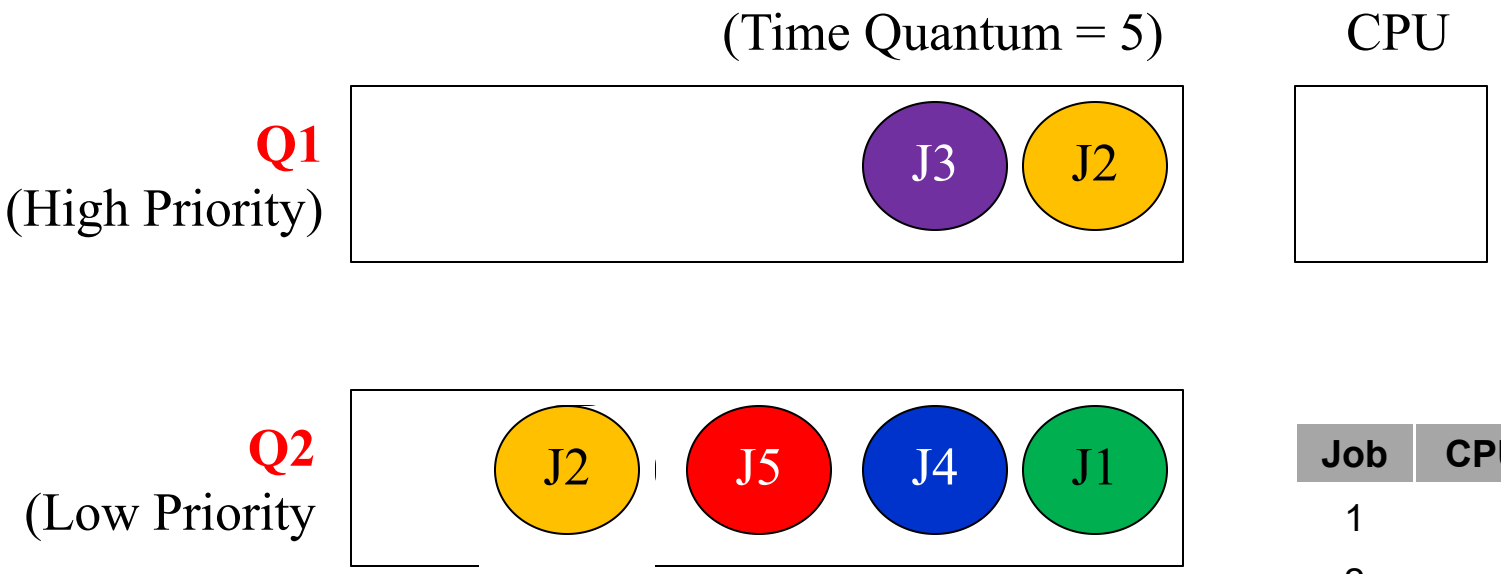
Case 2: Movement Between Queues



Job	CPU Burst
1	5
2	10
3	4
4	2
5	3

Arrival of jobs: J1, J2, J3, J4, J5

Case 2: Movement Between Queues



Job	CPU Burst
1	5
2	10
3	4
4	2
5	3

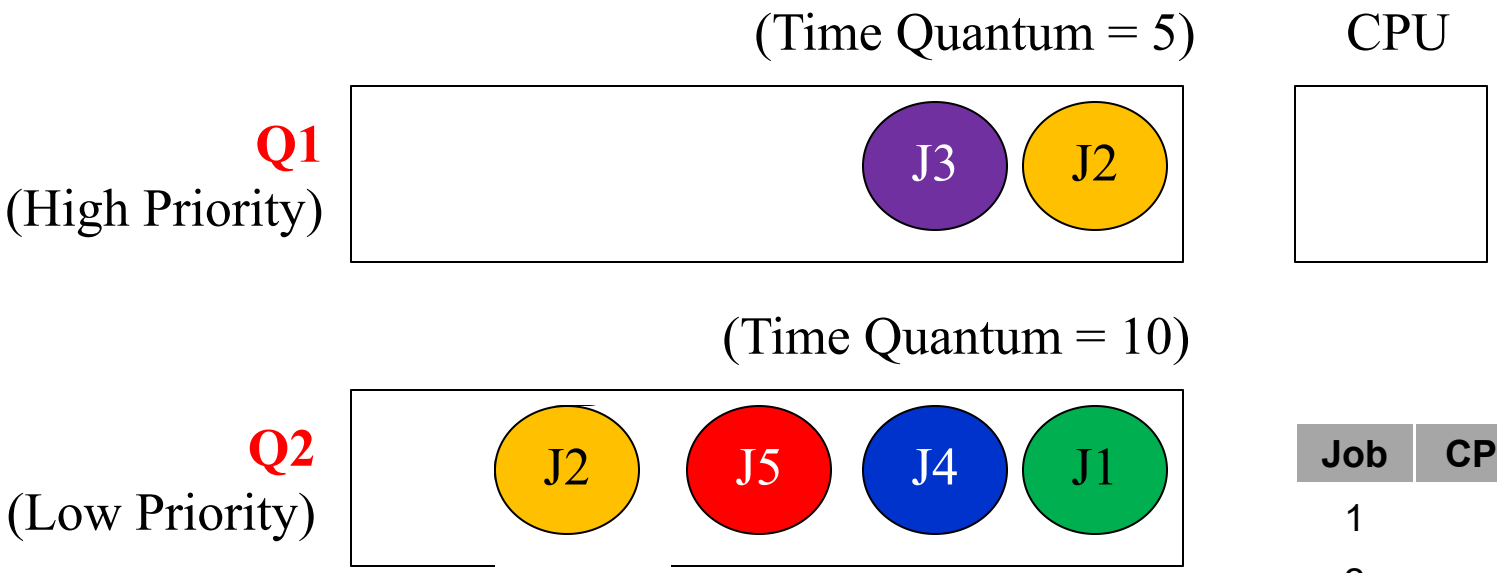
Case 3: Variable Time Quantum Per Queue

- Is a variation of the movement between queues
- Each queue given time quantum size
  - Size twice as long as previous queue
- Fast turnaround for CPU-bound jobs
- CPU-bound jobs execute longer and given longer time periods
  - Improves chance of finishing faster

I/O-bound jobs (such as printing a series of documents) have many brief CPU bursts and long I/O cycles,

CPU-bound jobs (such as finding the first 300 prime numbers) have long CPU bursts and shorter I/O cycles.

Case 3: Variable Time Quantum Per Queue

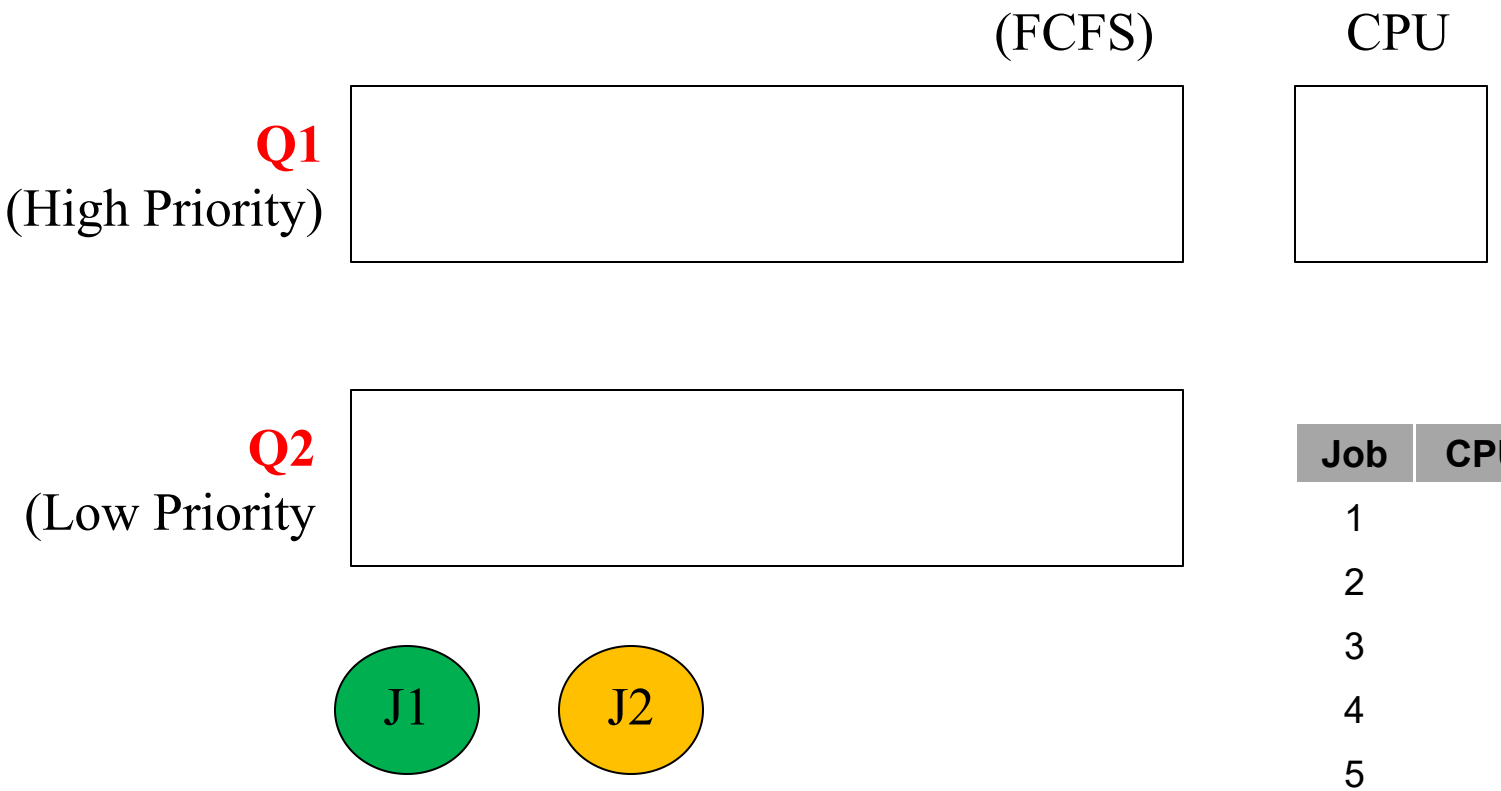


Job	CPU Burst
1	9
2	15
3	4
4	2
5	3

Case 4: Aging

- Ensures lower-level queue jobs eventually complete execution
- System keeps track of job wait time
- If too "old"
  - System moves job to next highest queue
  - Continues until old job reaches top queue
  - May drastically move old job to highest queue
- Advantage
  - Guards against indefinite unwieldy job postponement

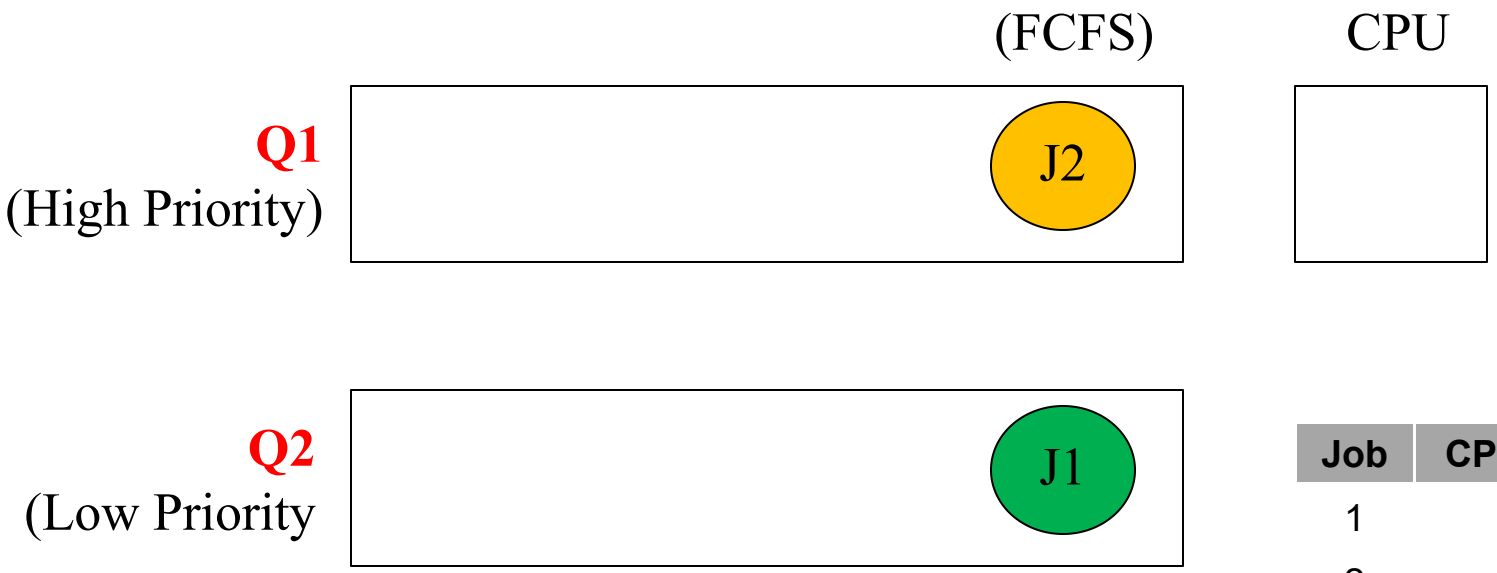
Case 4: Aging



Job	CPU Burst
1	9
2	100
3	30
4	20
5	5

Arrival of jobs at t0: J1, J2

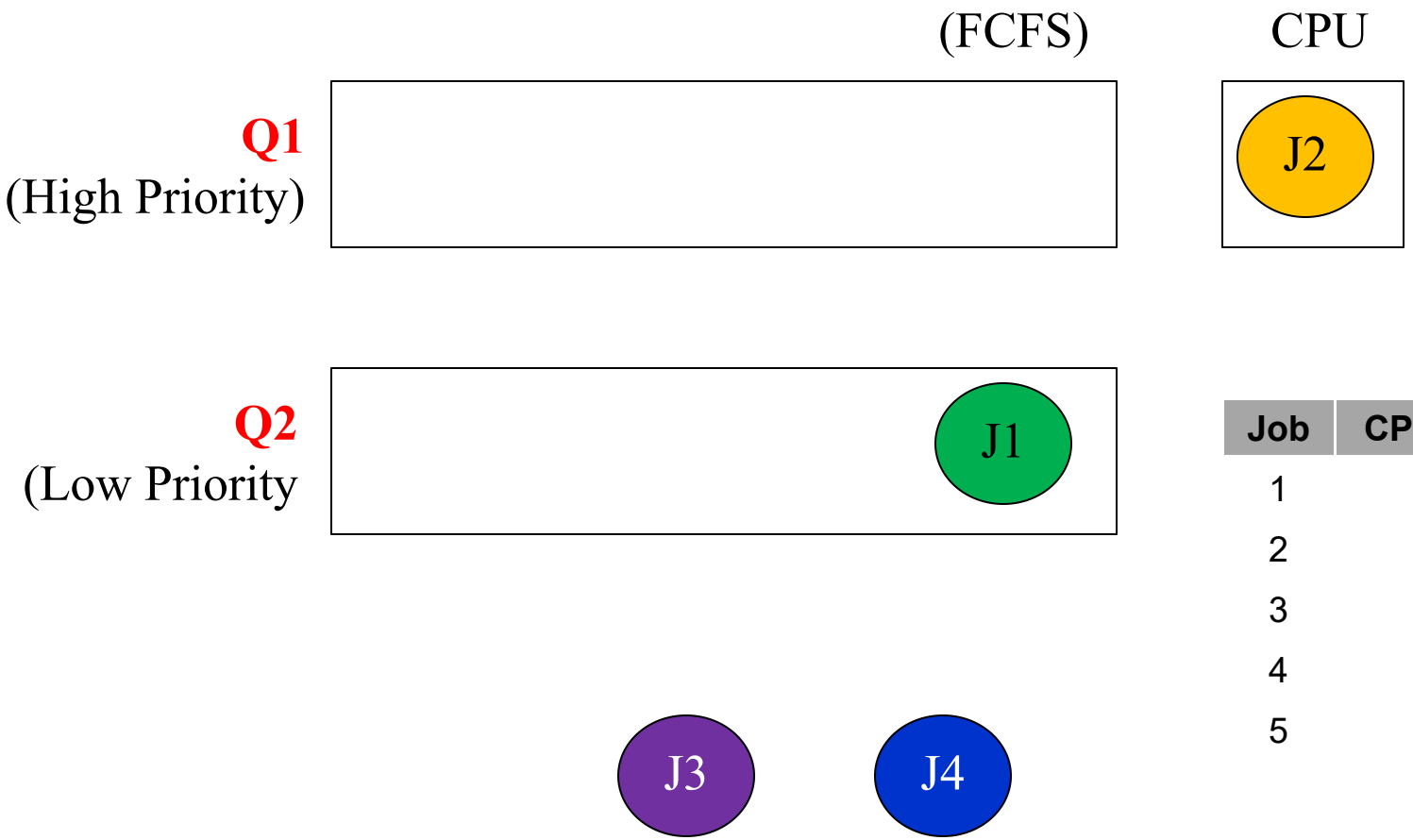
Case 4: Aging



Job	CPU Burst
1	9
2	100
3	30
4	20
5	5

Job 2 is running

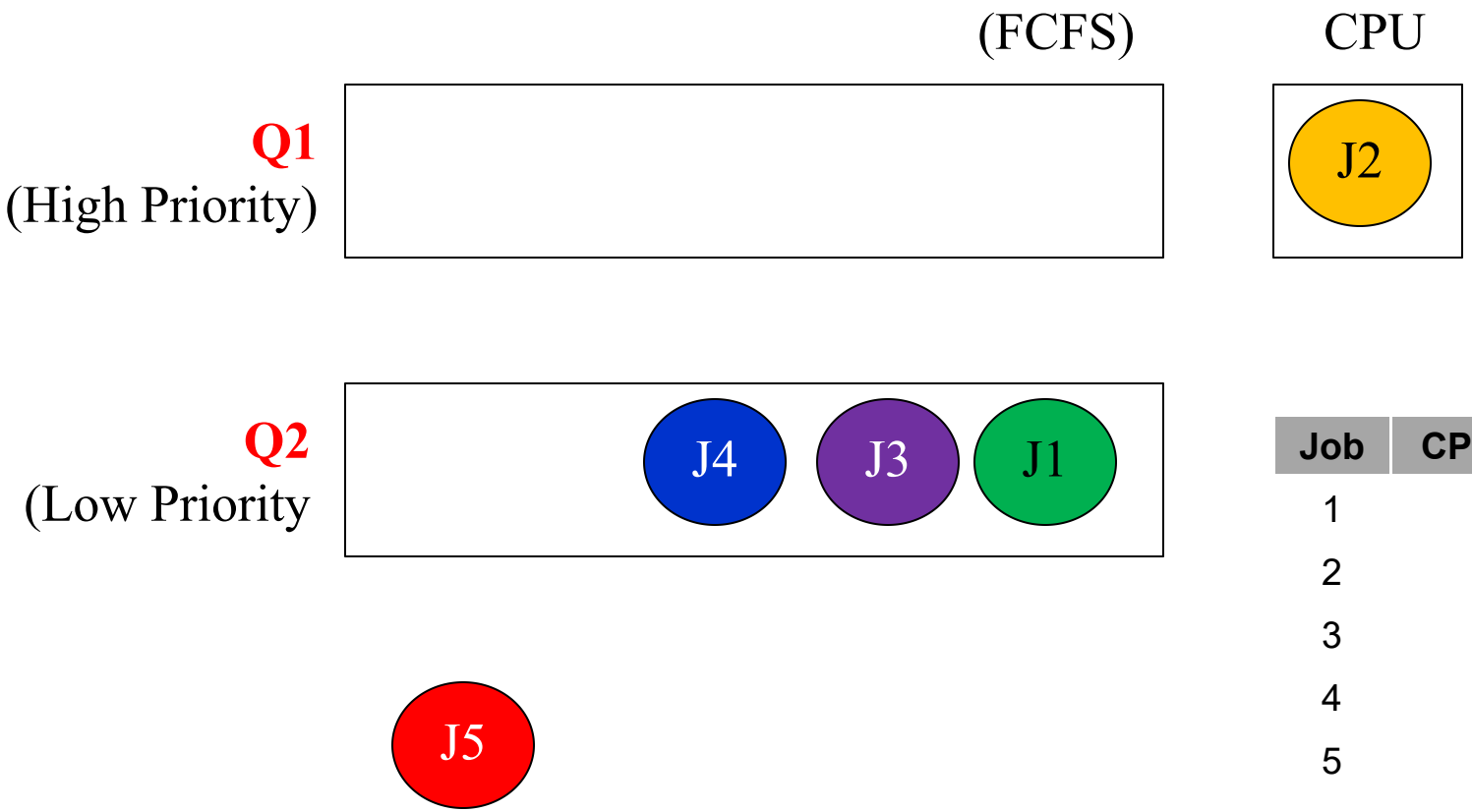
Case 4: Aging



Job	CPU Burst
1	9
2	100
3	30
4	20
5	5

t50 → Next new jobs arrived: J3, J4 (low priority), J2 still running

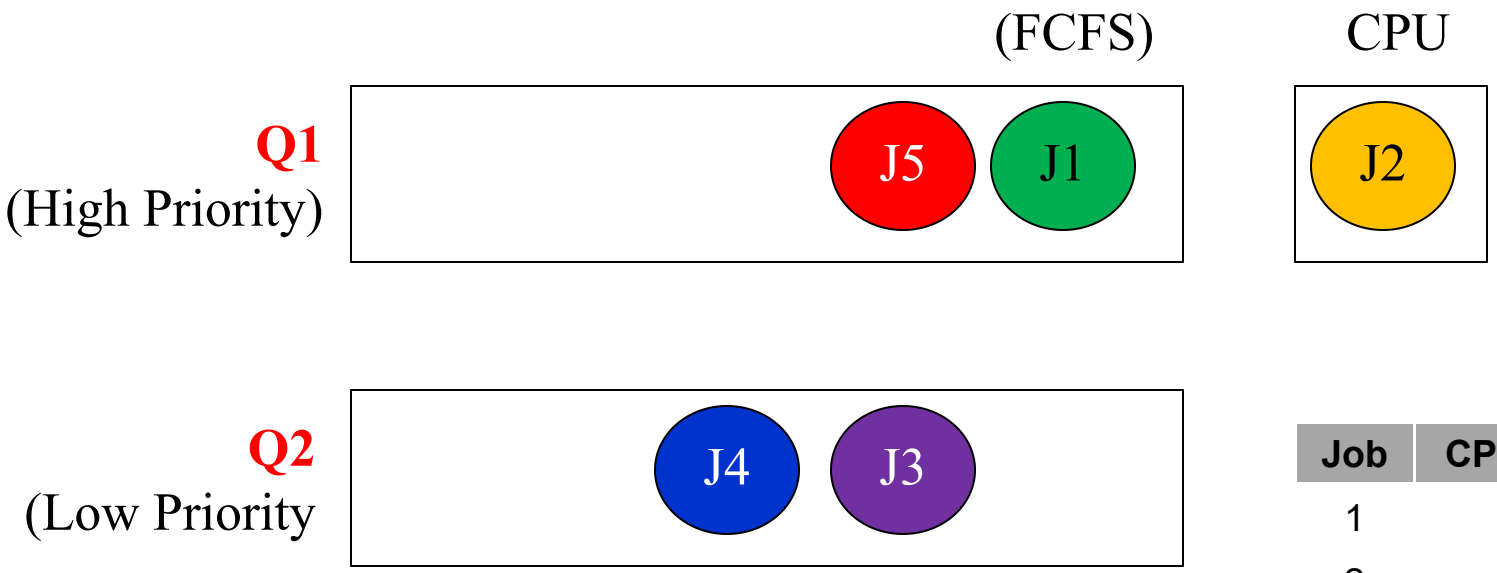
Case 4: Aging



Job	CPU Burst
1	9
2	100
3	30
4	20
5	5

t70 → Next new jobs arrived: J5 (High priority),
J2 still running, J1 has been waiting 70 CPU burst time

Case 4: Aging

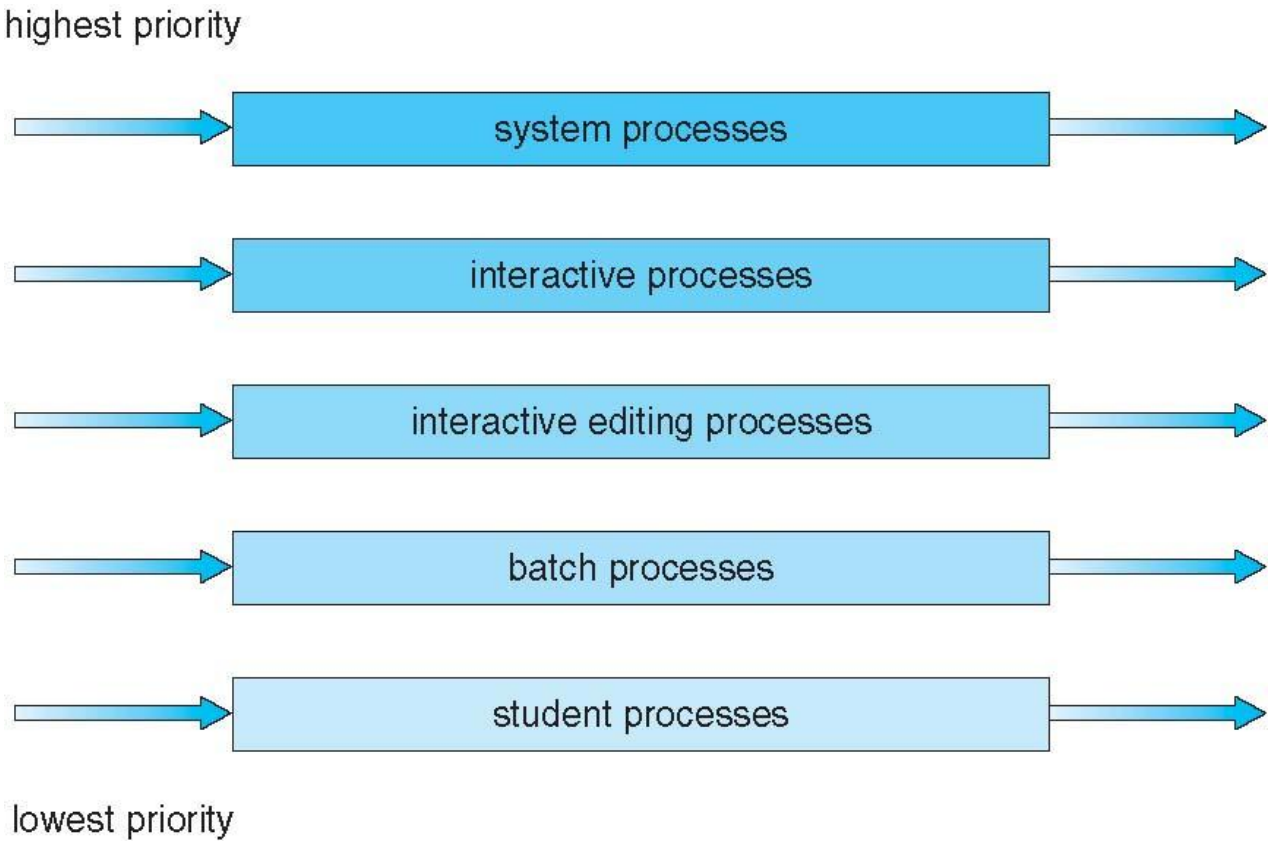


Job	CPU Burst
1	9
2	100
3	30
4	20
5	5

t100 → J2 , completed. J5 then running.
Continued with low priority jobs.



Multilevel Queue Scheduling





Algorithm	Policy Type	Best for	Disadvantages	Advantages
FCFS	Nonpreemptive	Batch	Unpredictable turnaround times	Easy to implement
SJN #SJF	Nonpreemptive	Batch	Indefinite postponement of some jobs	Minimizes average waiting time
Priority scheduling	Nonpreemptive #can be preemptive also	Batch	Indefinite postponement of some jobs	Ensures fast completion of important jobs
SRT	Preemptive	Batch	Overhead incurred by context switching	Ensures fast completion of short jobs
Round robin	Preemptive	Interactive	Requires selection of good time quantum	Provides reasonable response times to interactive users; provides fair CPU allocation
Multiple-level queues	Preemptive/ Nonpreemptive	Batch/ interactive	Overhead incurred by monitoring of queues	Flexible scheme; counteracts indefinite postponement with aging or other queue movement; gives fair treatment to CPU-bound jobs by incrementing time quantums on lower-priority queues or other queue movement

(table 4.1)

Comparison of the scheduling algorithms discussed in this chapter.



Multilevel Feedback Queue

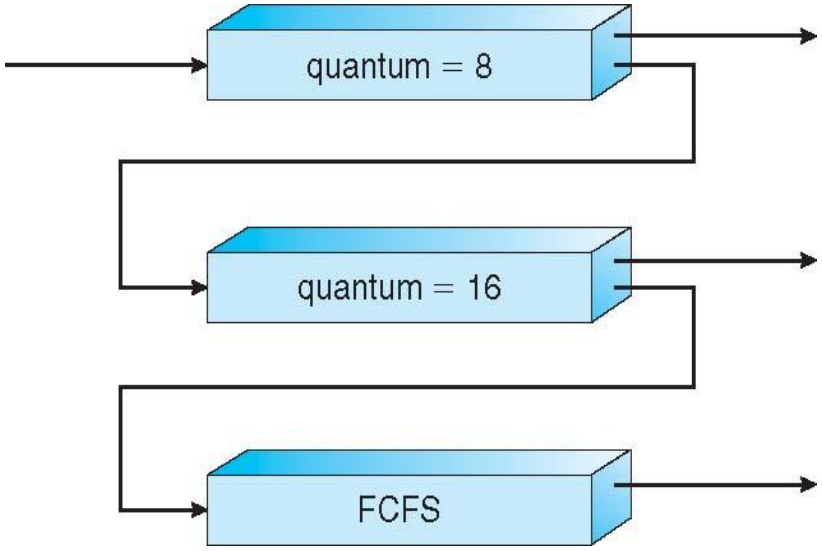
- A process can move between the various queues; aging can be implemented this way.
- The idea is to separate process according to the characteristics of their CPU bursts.
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service





Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS



- Scheduling
  - A new job enters queue Q_0 which is served FCFS
    - When it gains CPU, job receives 8 milliseconds
    - If it does not finish in 8 milliseconds, job is moved to queue Q_1
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds
    - If it still does not complete, it is preempted and moved to queue Q_2

- Processes in queue Q_2 are run on an FCFS basis but are run only when queues Q_0 and Q_1 are empty.
- This scheduling algorithm gives highest priority to any process with a CPU burst of 8 ms or less.

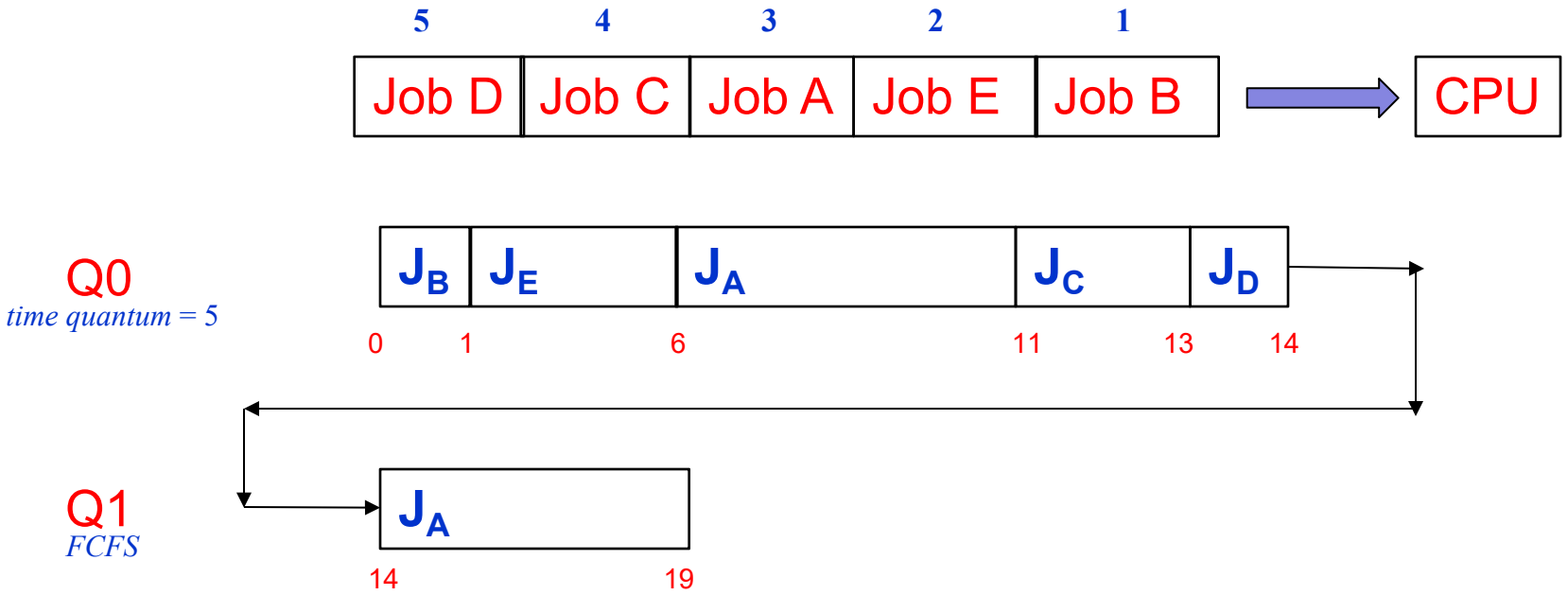


MFQ: Exercise 9

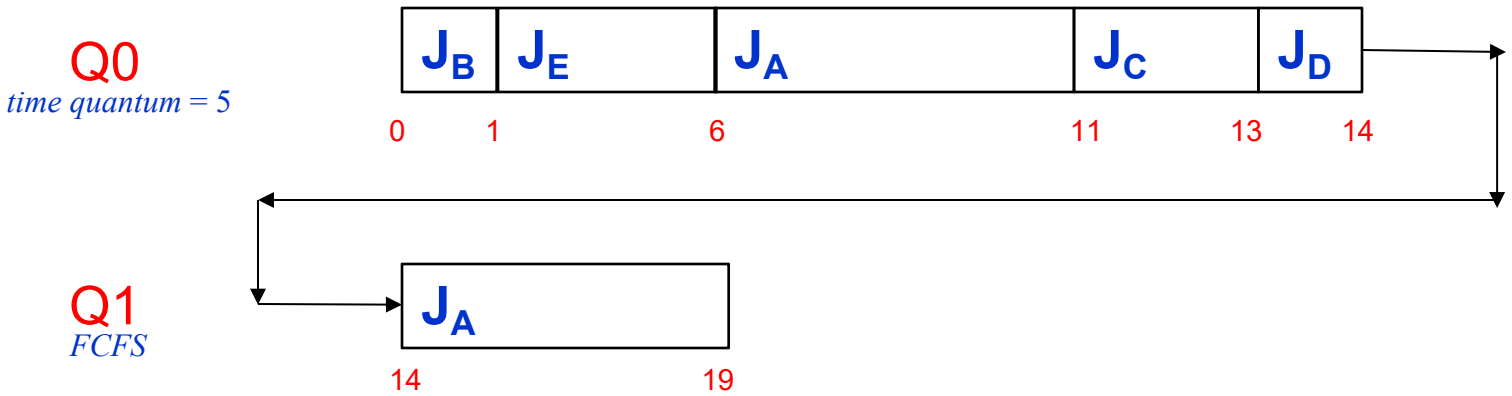
Five processes Job A, Job B, Job C, Job D and Job E arrived at time t_0 with different CPU burst and priority level. Assume that MFQ Scheduling algorithm is applied with 2 levels and the priority range 1 Highest. By given an initial time quantum for the first level is 5 CPU bursts and the last level using FCFS, calculate the average waiting time and average turnaround time.

Process	CPU Burst	Priority
Job A	10	3
Job B	1	1
Job C	2	4
Job D	1	5
Job E	5	2

Process	CPU Burst		Priority
Job A	~~10~~	~~5~~ 0	3
Job B	~~1~~	0	1
Job C	~~2~~	0	4
Job D	~~1~~	0	5
Job E	~~5~~	0	2



Process	CPU Burst	Priority	Completion	Turnaround time	Waiting time
Job A	10	3	19	19 – 0 = 19	(6 – 0)+(14 – 11) = 10
Job B	1	1	1	1 – 0 = 1	0 - 0 = 0
Job C	2	4	13	13 – 0 = 13	11 – 0 = 11
Job D	1	5	14	14 - 0 = 14	13 – 0 = 13
Job E	5	2	6	6 – 0 = 6	1 - 0 = 1



Average Turnaround Time = (20 + 1 + 13 + 1 4+ 6)/5 = 10.6 ms

Average Waiting Time = (10 + 0 + 11 + 13 + 1)/5 = 7 ms

EXERCISE (DIY!!)

The table below shows five jobs A, B, C, D and E arrived at different times and with different CPU bursts. The Multilevel Feedback Queue (MFQ) scheduling is used with three queues (Qi). Given an initial time, the quantum for the Q_0 is 5, Q_1 is 10 and the last queue uses FCFS. Calculate the following questions [12m]:

Jobs	Arrival Time	CPU Burst
A	0	15
B	4	35
C	26	4
D	28	24
E	32	33

i. What is the maximum value of CPU burst time that the jobs get at the Q1 level? [2]

ii. Which job(es) will finish at level Q1? [2]

iii. At the Q1 level, what is the second job that gets the CPU, and at what time does it occur? [2]

iv. What is the Waiting Time of Process B? [2]

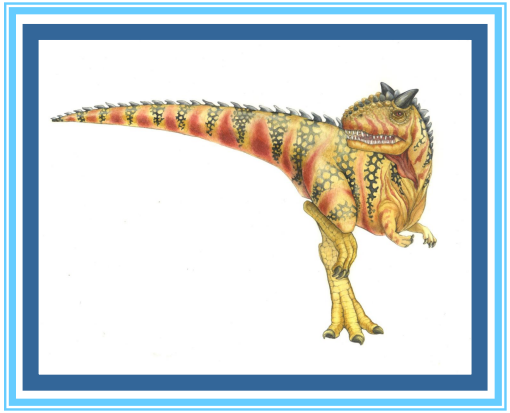
v. What is the Average Turnaround time? [2]

vi. What is the Average Waiting time? [2]

Summary

- Processor Manager allocates CPU among all users
- Job Scheduler
  - Assigns job to `READY` queue
    - Based on characteristics
- Process Scheduler
  - Instant-by-instant allocation of CPU
- Scheduling algorithm is unique
  - Characteristics, objectives, and applications
- System designer selects best policy and algorithm
  - After careful strengths and weaknesses evaluation

End of Chapter 5



Silberschatz, Galvin and Gagne ©2013